

审定成绩：_____

重庆邮电大学
毕业设计（论文）

中文题目	基于交互式需求澄清的智能代码生成系统 设计与实现
英文题目	Design and Implementation of an Intelligent Code Generation System Based on Interactive Requirement Clarification
学院名称	计算机科学与技术学院（示范性软件学院）
学生姓名	邹媛媛
专 业	软件工程
班 级	13002205
学 号	2022214193
指导教师	黄江平 讲师
答 辩 组 负 责 人	周俊 正高级工程师

二〇二六年五月

重庆邮电大学教务处制

计算机科学与技术学院（示范性软件学院）本科

毕业设计(论文)诚信承诺书

本人郑重承诺：

我向学院呈交的论文《基于交互式需求澄清的智能代码生成系统设计与实现》，是本人在指导教师的指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文不含任何其他个人或集体已经发表或撰写过的作品成果。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明并致谢。本人完全意识到本声明的法律结果由本人承担。

年级 2022 级

专业 软件工程

班级 13002205

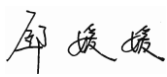
承诺人签名 

2026 年 5 月 18 日

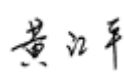
学位论文版权使用授权书

本人完全了解重庆邮电大学有权保留、使用学位论文纸质版和电子版的规定，即学校有权向国家有关部门或机构送交论文，允许论文被查阅和借阅等。本人授权重庆邮电大学可以公布本学位论文的全部或部分内容，可编入有关数据库或信息系统进行检索、分析或评价，可以采用影印、缩印、扫描或拷贝等复制手段保存、汇编本学位论文。

（注：保密的学位论文在解密后适用本授权书。）

学生签名： 

日期： 2026 年 5 月 18 日

指导老师签名： 

日期： 2026 年 5 月 18 日

摘要

大语言模型驱动的智能代码生成（如 GitHub Copilot）已显著提升开发效率，但单次提示生成方式在处理模糊或不完整需求时易产生错误，需求模糊性成为其生产落地的关键障碍。

本文基于交互式需求澄清思路，围绕模糊需求识别、澄清问题生成与多轮对话交互机制，开展融合设计与实现。主要工作包括：

第一，需求模糊性分析与流程设计：分析模糊需求对代码生成质量的影响，论证交互澄清的必要性；引入“人在回路”模式，构建“检测—提问—整合—生成”的人机协同工作流，完成可行性分析与总体设计。

第二，模糊性检测模块：利用大语言模型的需求分析能力，设计两阶段提示策略，从输入规格、边界条件、异常处理等维度自动识别缺失关键信息。

第三，多轮对话管理与澄清生成：设计基于有限状态机的对话管理模块与澄清问题生成模块，通过维护对话状态与结构化需求规约，实现主动提问与用户反馈的动态整合，逐步完善需求。

第四，结构化规约的代码生成与评估：设计代码生成与解释模块，在需求澄清后输出高质量代码及自然语言解释；构建典型模糊场景测试集，设计对比实验，以 Pass@1 为评估指标。

实验结果表明：在含 10 个模糊需求的测试集上，经交互澄清后，代码通过率（Pass@1）从直接生成的 70.0% 提升至 100.0%，绝对提高 30 个百分点（相对提高 42.9%）。该机制提升了生成代码的功能正确性，为 AI 编程工具跨越“语义鸿沟”提供了可行路径。

关键词：交互式需求澄清；智能代码生成；大语言模型；人在回路；人机协同

Abstract

Large language model-driven intelligent code generation (e.g., GitHub Copilot) has significantly improved development efficiency. However, the single-turn prompt generation approach is prone to errors when handling ambiguous or incomplete requirements, making requirement ambiguity a key obstacle to production deployment.

Based on the idea of interactive requirement clarification, the integrated design and implementation of ambiguous requirement identification, clarification question generation, and multi-turn dialogue interaction mechanisms are studied. The main contributions are as follows:

1. Requirement ambiguity analysis and workflow design: Analyzes the impact of ambiguous requirements on code generation quality and demonstrates the necessity of interactive clarification. Introduces the human-in-the-loop paradigm and constructs a “detection–questioning–integration–generation” human-machine collaborative workflow, followed by feasibility analysis and overall system design.

2. Ambiguity detection module: Leverages the requirement analysis capability of large language models to design a two-stage prompting strategy that automatically identifies missing critical information from dimensions such as input specifications, boundary conditions, and exception handling.

3. Multi-turn dialogue management and clarification generation: Designs a finite state machine-based dialogue management module and a clarification question generation module. By maintaining dialogue state and structured requirement specifications, the system achieves proactive questioning and dynamic integration of user feedback, gradually refining the completeness and clarity of requirements.

4. Structured specification-based code generation and evaluation: Designs a code generation and explanation module that produces high-quality code along with natural language explanations after requirement clarification. Constructs a test dataset covering typical ambiguous scenarios and designs comparative experiments using Pass@1 as the evaluation metric.

Experimental results show that on a test set containing 10 ambiguous requirements, after interactive requirement clarification, the Pass@1 code pass rate improves from 70.0% (direct generation) to 100.0%, an absolute increase of 30 percentage points

(relative increase of 42.9%). This mechanism significantly enhances the functional correctness of generated code and provides a feasible path for AI programming tools to bridge the “semantic gap” in practical applications.

Keywords: Interactive requirement clarification; Intelligent code generation; Large language model; Human-in-the-loop; Human-machine collaboration

目录

第 1 章 引言	1
1.1 研究背景及意义	1
1.1.1 研究背景	1
1.1.2 研究意义	2
1.2 国内外研究现状	3
1.2.1 HITL 研究现状	3
1.2.2 交互式需求澄清研究现状	3
1.2.3 大型语言模型（LLM）研究现状	5
1.3 研究内容与论文结构	6
1.3.1 研究内容	6
1.3.2 论文结构	7
第 2 章 相关技术综述	9
2.1 大语言模型驱动下的代码生成技术	9
2.1.1 大语言模型的发展与代码能力的涌现	9
2.1.2 训练策略与解码优化	9
2.1.3 模糊需求下的代码生成挑战	10
2.2 交互式需求澄清技术	10
2.2.1 需求工程中的模糊性管理	10
2.2.2 基于对话的交互式需求获取	11
2.2.3 对话式澄清中的问题生成策略	11
2.2.4 结构化需求澄清框架：ChatCoder 与 ClarifyGPT	12
2.2.5 历史经验学习与案例推理	14
2.2.6 交互式需求澄清中的可解释性与透明性	14
2.3 自动化测试与代码质量评估	14
2.3.1 基于 LLM 的自动化测试生成	14
2.3.2 安全沙箱与代码执行隔离	15

2.3.3 代码质量的多维度评估.....	15
2.4 多智能体协作与人在回路.....	16
2.4.1 多智能体分工协作架构.....	16
2.4.2 人在回路的设计原则与交互准则.....	16
2.4.3 反馈驱动模型对齐与持续优化.....	17
2.5 对话状态管理与交互策略.....	17
2.5.1 任务型对话系统的形式化.....	17
2.5.2 对话状态的持久化与恢复.....	17
2.5.3 交互策略的用户适应性.....	18
2.6 系统架构与工程实现.....	18
2.6.1 FastAPI 框架与 RESTful API.....	18
2.6.2 数据库设计与多用户数据隔离.....	18
2.6.3 身份认证与会话安全.....	19
2.6.4 前端交互与可视化.....	19
2.7 本章小结.....	20
第 3 章 系统需求分析.....	21
3.1 需求分析.....	21
3.2 功能性需求分析.....	22
3.2.1 业务用例分析.....	22
3.2.2 系统用例分析.....	23
3.2.3 用例规约.....	25
3.2.4 系统用例活动图.....	29
3.3 非功能性需求分析.....	30
3.3.1 性能需求.....	30
3.3.2 安全需求.....	30
3.3.3 可靠性需求.....	30
3.3.4 可扩展性需求.....	31
3.4 约束需求.....	31
3.5 本章小结.....	31

第 4 章 系统概要设计	33
4.1 系统总体架构设计	33
4.2 系统功能模块设计	34
4.3 系统工作流程	36
4.4 对话管理状态机设计	37
4.5 数据库设计	39
4.5.1 概念模型设计	39
4.5.2 数据库表设计	39
4.6 本章小结	44
第 5 章 系统详细设计及实现	45
5.1 多智能体协作框架的实现	45
5.2 需求模糊性检测模块（分析智能体）	45
5.2.1 模块设计原理	45
5.2.2 模块实现	46
5.3 澄清问题生成模块（问题生成智能体）	48
5.3.1 模块设计原理	48
5.3.2 模块实现	49
5.4 多轮对话管理与回滚机制（协调器+回滚智能体）	51
5.4.1 对话状态管理	51
5.4.2 答案规范化处理	52
5.4.3 回滚机制	53
5.4.4 状态持久化与会话恢复	55
5.5 历史经验学习机制（历史经验智能体）	56
5.5.1 机制设计原理	56
5.5.2 模块实现	57
5.6 代码生成与安全检测模块（代码生成智能体）	58
5.6.1 模块设计原理	58
5.6.2 模块实现	58
5.6.3 安全检测	61

5.7 前端交互实现与完整功能界面	62
5.7.1 核心交互逻辑	62
5.7.2 用户认证与系统入口	63
5.7.3 历史任务与规格说明书管理	65
5.7.4 代码分析可视化	66
5.7.5 高质量案例库	67
5.7.6 技能管理	68
5.7.7 账号管理	69
5.7.8 模型配置与用户管理（管理员专属）	70
5.8 结构化需求规约示例	71
5.9 本章小结	72
第 6 章 系统测试与评估	73
6.1 测试需求分析	73
6.2 实验环境与配置	73
6.3 单元测试	74
6.4 集成测试	75
6.5 系统测试	76
6.5.1 功能测试	76
6.5.2 代码通过率验证（对比实验）	81
6.5.3 性能测试	84
6.6 测试结果与结论	84
6.7 本章小结	84
第 7 章 总结与展望	85
7.1 主要工作	85
7.2 工作展望	86
参考文献	87
致谢	93

第 1 章 引言

1.1 研究背景及意义

1.1.1 研究背景

近年来，依靠大语言模型的智能代码生成技术有了飞速的发展。以 OpenAI Codex、GitHub Copilot 为代表的产品，通过大量代码数据进行预训练，可以将自然语言描述直接转化为可执行的代码片段，这是人工智能辅助软件开发的一个新的阶段^[1]。这类工具在一定程度上提高了程序编写效率，对于明确且简单的任务具有较好的表现。

但是目前主流的“单次提示生成”（One-Shot Prompting）模式对于真实世界中大量的模糊、不完整或者有歧义的自然语言需求，存在着根本性的不足。其原因是自然语言提示作为输入接口，本身具有模糊性、非结构化和上下文依赖性的特点，并作用于非确定性的模型^[2]。例如，用户提出“写一个排序函数”这一需求，其中并未明确边界条件、输入输出格式或性能约束。在此情况下，大语言模型只能根据训练数据的统计模式进行概率性推测，可能生成与用户意图不符、含有逻辑错误或无法编译的代码。有研究发现，开发者在理解、修改和调试模型生成的代码时面临较大困难，进而影响任务完成的效率与成功率^[3]。因此，需要解决由需求模糊性和模型概率性推测所导致的问题，从需求源头提高生成代码的准确性和可用性，这是该技术从演示场景走向实际应用过程中需要面对的课题。

在此背景下，“人在回路”（Human-in-the-Loop, HITL）的交互式理念给解决这个问题提供了一种思路。参照人类软件开发中需求分析师和开发者用问答的形式来明确细节的做法，交互式需求澄清把用户被动接受结果的模式转变为系统主动引导、用户不断反馈的协同创作闭环^[4]。通过构建一个能够自动识别需求模糊点、智能生成澄清问题并管理多轮对话的系统，我们有望在代码生成前对齐人机意图，从而提升最终代码的质量。本文正是在该技术发展趋势下提出的实践探索。

1.1.2 研究意义

大语言模型在软件工程领域的应用逐渐广泛，对代码生成等相关任务产生了一定影响。随着模型能力的提升和应用场景的扩展，智能代码生成技术已用于需求分析、代码编写和软件测试等环节。在信息技术快速发展的背景下，软件工程领域正朝着智能化、自动化的方向演变。开发并应用智能代码生成系统，符合人工智能与软件工程相结合的发展趋势，有助于提高软件开发的效率和质量，对降低开发成本、提升软件可靠性具有实际意义^[5]。

在智能代码生成的实际使用过程中，开发者常遇到模糊、不完整或有歧义的自然语言需求。这类需求导致开发者难以准确理解用户意图，调试生成的代码也较为困难。通过与开发者交流与调研，发现该问题具有一定普遍性。对代码生成任务的全流程进行分析后，可总结出以下三个主要问题：

第一，需求的表达不够清晰。用户难以一次性完整、准确地描述所有功能细节，模型因此难以完全理解用户真实意图，生成的代码容易产生逻辑错误或功能缺失。有研究表明，开发者在理解、编辑和调试模型生成代码时存在较大困难，影响了任务完成的效率与成功率^[3]。

第二，大语言模型对自然语言的解析依赖于训练数据中的统计规律。当遇到如“写一个排序函数”这类未明确边界条件或性能约束的模糊需求时，模型只能基于训练数据的统计模式进行概率性推测，较易生成不符合用户意图的代码^[2]。

第三，在交互方式上，现有工具缺乏有效的反馈与澄清机制。用户无法在生成过程中引导模型理解隐含意图，只能被动接受结果后再反复调试和修改，沟通成本较高，开发效率较低，未能充分发挥 AI 辅助编程的作用。

本次毕业设计针对上述问题，将人机交互与自然语言处理技术结合应用于智能代码生成中，采用交互式需求澄清机制来缓解需求模糊性问题。具体方法包括：使用大语言模型识别需求中缺失的信息并主动提问，引导用户完善需求描述；将用户的补充回答动态整合为结构化的需求规约，使模型能够在明确需求的基础上生成代码。基于此，设计并实现一个交互式的代码生成智能需求澄清系统，以改善智能代码生成过程中需求模糊的问题，优化人机协作方式，更好地辅助软件开发。

1.2 国内外研究现状

1.2.1 HITL 研究现状

HITL 是人机协同的一种常见范式。当前，该范式的应用场景正在从单个机器学习任务的优化扩展到复杂系统工程的支持。现有研究不再仅将人类视为数据标注员或结果校验员，而是尝试在动态、不确定的环境中构建人类与系统之间较为紧密的协作关系^[6]。在智能代码生成等专业领域，系统需要理解用户的模糊意图，并主动引导用户澄清需求，吸收领域知识，通过持续协作完成代码生成任务。

在研究层面，相关工作涉及多个方面。系统架构与设计原则的研究重点是将人类因素纳入信息物理系统（CPS）等复杂工程中。这要求在软件工程层面不仅考虑技术实现，还需从社会技术角度考虑人类在环境感知、基础设施交互、多方协调和最终决策中的作用。HITL 理念已应用于机器学习的多个环节，例如在数据预处理阶段利用人类知识进行标注和清洗，在模型训练阶段依据人类反馈进行调整，在决策推理阶段由人类审核关键输出。在数据融合方面，HITL 机制允许用户对实体匹配、聚类纠错、数据转换等环节的反馈意见进行持续修正，从而提高系统的准确性和稳定性^[7]。此外，大语言模型的出现使得 HITL 研究开始采用人机回路与模型回路相结合的方式，将大模型作为中间件连接人与复杂系统，形成完整的交互闭环^[8]。

在应用领域方面，HITL 已从传统领域扩展到智能制造、智慧医疗、自动驾驶以及个性化教育等场景。在这些高风险或强交互的领域中，系统的正常运行依赖于人类智能对动态环境和模糊边界的实时理解与干预。在智能代码生成方向，HITL 理念被用于构建交互式编程助手，通过多轮对话主动澄清用户需求，提高生成代码的质量和可用性。

1.2.2 交互式需求澄清研究现状

交互式需求澄清用于连接用户的模糊意图与系统的代码生成结果，是智能软件工程领域的一个研究方向。该方向关注的问题是如何让大语言模型等系统从被动接收需求转向主动与用户对话，通过多轮交互理解并获取用户的真实需求。

当前研究趋势表明，研究重点已从单一技术点的突破转向系统化、可学习的框架构建。具体而言，可归纳为以下三个方面。

第一，需求模糊性的识别方法。已有研究尝试建立系统的分类标准。AmbiTRUS 从词汇、句法、语义和语用四个语言学角度识别用户故事中隐含的模糊问题，并给出了一套分类方法与质量评估标准^[9]。

第二，澄清交互的策略。现有研究从关注问题生成转向关注交互过程中的信息透明和情境共享。有研究发现，在人机协作过程中，向用户提供适度的上下文信息（例如让用户了解系统如何理解其意图）有助于提高协作效率和用户体验；这表明透明度和共享情境感知在交互式澄清中具有一定价值^[10]。

第三，基于大语言模型的交互澄清智能体。随着大语言模型能力的提升，相关研究开始构建端到端、可训练的交互澄清智能体。基于强化学习的 UserRL 框架将多轮交互抽象为序列决策问题，使模型自主学习何时提问以及如何澄清，从而逐步优化交互策略^[11]。

在理论层面，交互式需求澄清的深层机理可追溯至认知科学和语用学中关于对话协同的经典理论。Clark 与 Schaefer 提出的贡献模型指出，对话双方通过“呈现—接受”的协同循环逐步建立共同基础，每个话轮都对共享知识库进行增量式更新^[12]。将这一理论映射到人机澄清场景，系统生成问题即为呈现澄清请求，用户提供信息则为接受并扩展共同基础，随后系统还需通过复述或确认来巩固这一基础，从而形成完整的协同闭环。这一过程要求系统能够准确判断当前共同基础的缺口，并生成精准获取缺失信息的提问。

Grice 的会话合作原则为澄清问题的设计提供了重要的语用规范^[13]。根据质量准则，系统生成的问题应基于对需求缺失的真实判断，不应询问已知或无关的信息；数量准则要求单轮提问所承载的信息量适中，避免将多个不相关问题杂糅在一起而增加用户的认知负荷；关系准则强调问题须紧扣当前需求缺口，不偏离主题；方式准则则要求问题表述清晰、无歧义，避免因提问方式的模糊造成二次误解。遵循这些准则，可使澄清对话在信息效率与交互自然度之间取得平衡。

在需求模糊性分类方面，研究者将自然语言需求中的模糊现象进一步细分为指示模糊、范围模糊、情态模糊等多种类型。不同类型的模糊信息缺失模式存在差异，因而需要设计差异化的提问策略。例如，对于“合适的”“快速的”等指

示模糊，可要求用户给出可量化的判定标准；对于“所有用户”“部分数据”等范围模糊，可追问具体包含或排除的边界条件。这为本文设计基于模糊模式检查清单的结构化提问提供了直接的理论依据。此外，Sperber 与 Wilson 的关联理论从认知语用角度指出，交流双方倾向于以最小的加工努力获取最大的语境效果^[14]。这对澄清系统的启示是，应当在每个对话轮次中优先选择能够最大程度消除不确定性的问题，使用户能以最小的认知成本提供最关键的信息。上述理论共同构成了本文设计需求澄清引擎中模糊点检测、问题生成和对话管理策略的理论基础。

1.2.3 大型语言模型（LLM）研究现状

目前，以语言模型为对象的研究正从扩大规模转向提升能力。过去一年里，该领域的一个明显变化是从基于数据驱动的概率文本生成，转向具备逻辑推理能力的推理模型。这一变化源于基于验证的强化学习方法。在数学、代码等可自动验证结果的任务中训练模型，使模型生成中间推理步骤，从而形成类似人类的思维链^[15]。以 OpenAI 的 o1 系列和 DeepSeek-R1 为代表的新一代模型能够处理复杂数学证明、代码修复和战略规划等任务，扩展了语言模型的应用范围。与此同时，针对模型规模增大带来的计算成本问题，硬件、算法和软件栈方面的效率优化也在同步推进。例如，采用更高效率的计算格式设计芯片架构，使用推测解码、KV 缓存优化等软件算法，在提升模型能力的同时降低了训练和推理的能耗与延迟。

在模型架构方面，研究持续寻求对现有 Transformer 框架的改进。谷歌 DeepMind 等机构提出的混合递归结构，对不同 Token 分配不同的计算深度，从而提高推理速度并减少内存占用。学术界和工业界普遍认为，仅依靠增加参数和数据规模的扩展方式已面临边际效益递减，未来的进展将更多来自算法和架构层面的创新。此外，如何使模型更高效地适应不同任务也是研究的关注点。Emory 大学的研究表明，利用记忆机制和动态计算资源分配处理相似任务时，大型语言模型可以达到类似人类“熟能生巧”的效果，推理成本降低 56%^[16]。麻省理工学院提出的自适应语言模型框架，则探索让模型通过生成自身的微调数据和更新指令来适应新任务，展示了模型自我演化的可能方向。

1.3 研究内容与论文结构

1.3.1 研究内容

本次毕业设计的目标是，针对大语言模型在处理模糊需求时生成代码质量偏低的问题，通过引入人在回路机制，设计并实现一个基于交互式需求澄清的智能代码生成原型系统。研究的内容可以分成以下几个部分，如图 1.1 所示。

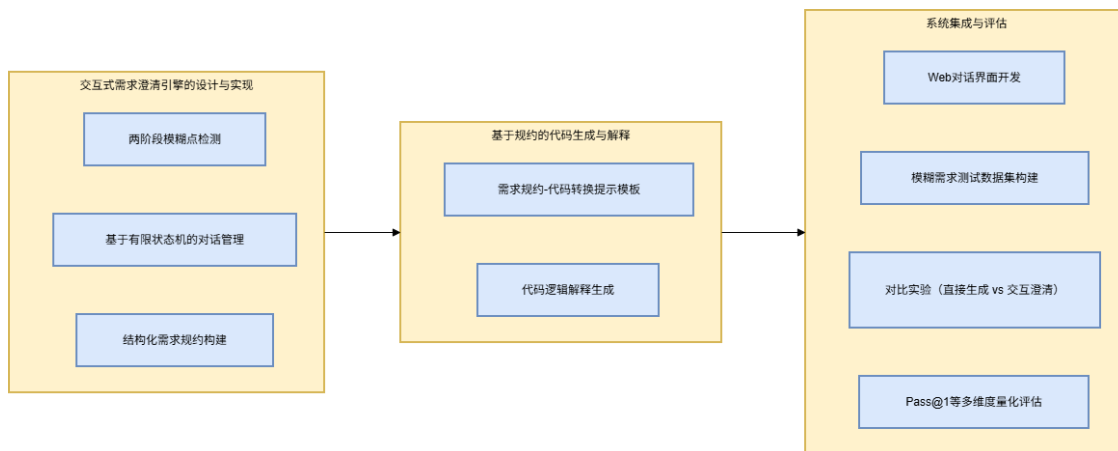


图 1.1 毕业设计研究内容图

第一，交互式需求澄清引擎的设计与实现。该部分研究大语言模型如何获取需求分析的能力。具体工作包括设计一个两阶段的提示策略，引导大语言模型检测用户初始需求中的模糊点：第一阶段进行开放式反思，列出可能的疑问点；第二阶段参照预先定义的模糊模式检查清单（包括输入输出规格、边界条件、异常处理、性能约束等）进行结构化核对，以保证覆盖的完整性。同时，设计并实现一个基于有限状态机的对话管理器，用于维护多轮交互过程中的上下文，并将用户反馈动态且一致地整合到原始需求中，逐步构建结构化的需求规约。

第二，基于规约的代码生成与解释。该部分研究如何利用大语言模型将澄清后的结构化需求规约转换为代码及相应的解释。内容包括优化专门用于“需求规约到代码”转换任务的提示模板，并研究如何生成简洁且聚焦的代码逻辑解释，以提高系统的透明度和用户对系统的信任。

第三，系统集成与评估。将澄清引擎、代码生成器等模块集成，形成一个带有网页对话界面的可运行系统。通过构建包含典型模糊场景的测试数据集，设计

对比实验（对照组采用直接生成方式，实验组在交互澄清后生成代码），从代码通过率（Pass@1）和需求完整性等方面量化评估系统的有效性。

1.3.2 论文结构

整篇论文由七个章节组成，论文章节结构如图 1.2 所示。

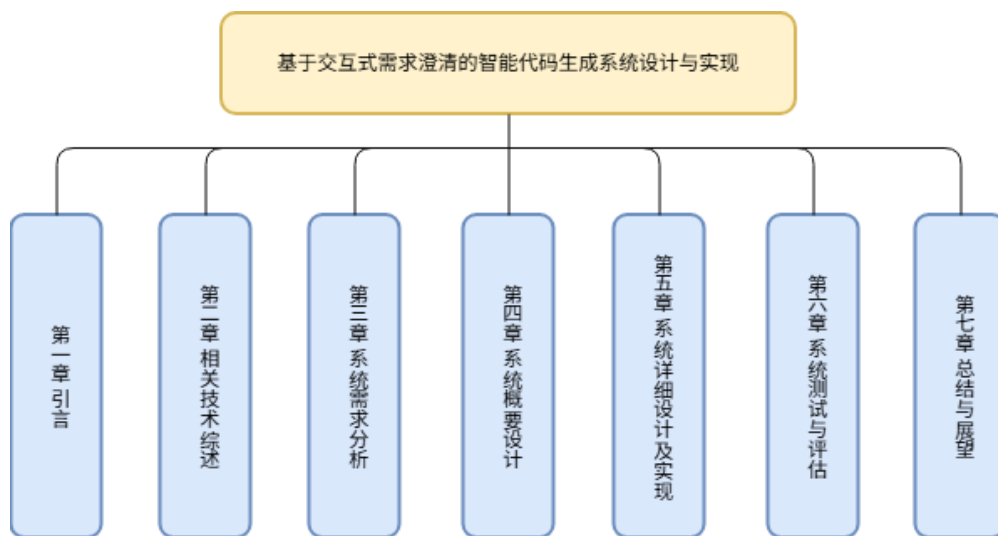


图 1.2 论文章节结构图

第一章是引言，简述选题的研究背景、研究意义，梳理国内外相关研究现状，并给出本文的主要研究内容与论文结构安排。

第二章是相关技术综述，系统综述三大核心技术：大语言模型与代码生成、交互式需求澄清技术、人在回路与对话系统，为后续系统设计奠定理论与技术基础。

第三章是系统需求分析，从需求分析出发，明确系统的功能性需求、非功能性需求以及约束需求。

第四章是系统概要设计，给出系统的总体架构设计、功能模块划分、核心工作流程、对话管理状态机设计，以及数据库的概念模型、表结构与表间关系。

第五章是系统详细设计及实现，逐一阐述多智能体协作框架下的各模块设计与实现细节，以及前端交互界面与结构化需求规约示例。

第六章是系统测试与评估，设计单元测试、集成测试、系统测试等多种测试，验证系统功能，通过对比实验评估 Pass@1 指标，分析需求完整性及典型用例，证明交互式需求澄清机制的有效性。

第七章是总结与展望，介绍本次毕业设计的主要工作，并对接下来工作可以改进、优化的地方进行展望。

第 2 章 相关技术综述

2.1 大语言模型驱动下的代码生成技术

2.1.1 大语言模型的发展与代码能力的涌现

基于自注意力机制的 Transformer 架构成为了现代自然语言处理的基石^[17]。在此基础上，以大量无监督文本预训练为基础的语言模型（GPT 系列）得到了广泛应用。GPT-1 验证了生成式预训练的有效性^[18]；GPT-2 揭示了零样本多任务能力^[19]；GPT-3 通过 1750 亿参数和上下文学习，在大量任务上无需微调即可取得较好性能^[20]。专门用于代码生成的大语言模型在此基础上快速发展。Chen 等人用 GPT-3 在 159GB 的 Python 代码数据集上微调得到 Codex，在 HumanEval 基准上取得了 28.8% 的 Pass@1^[21]。GPT-4 在 HumanEval 上的 Pass@1 进一步跃升至 67%^[22]，并被研究者评价为“展现出通用人工智能的火花”。开源领域同样涌现出 StarCoder、Code Llama、DeepSeek-Coder 等高性能模型，形成了多模型共存的生态。

在生成机制上，所有大语言模型均采用自回归解码方式，根据输入需求逐令牌预测输出代码序列。这种生成方式能够自然建模代码的层次结构，但也容易在生成长序列时累积误差。本系统的 llm_clients.py 模块通过工厂模式统一封装了 DeepSeek、Doubao 及 Qwen 兼容接口等多种模型后端，用户可以根据需求选择不同的模型。

2.1.2 训练策略与解码优化

代码生成模型的性能不仅取决于参数规模，还受益于训练策略的改进。填空中间（Fill-in-the-Middle, FIM）训练将代码补全任务建模为预测被遮蔽的中间部分，提升了 IDE 中实时代码补全的实用性。指令微调通过构造自然语言指令-代码响应对，使模型能更准确地理解用户意图。此外，重排序技术通过训练一个奖励模型对多个生成候选评分，然后选取最高分输出，已在代码生成中被证明有效^[23]。本系统在代码生成时采用 temperature=0.2 的确定性解码，并通过同时生成实验组

（澄清后）和对照组（无澄清）代码，然后进行测试通过率对比，得到类似多候选评估的效果。

2.1.3 模糊需求下的代码生成挑战

尽管 LLM 对于定义明确的任务表现良好，但现实世界的需求往往是不完整的、模糊的甚至是相互矛盾的。多项实证研究证明了这一点。Vaithilingam 等人通过用户实验发现，使用 AI 代码助手的开发者需要花费大量时间理解、修改和调试生成的代码，根本原因在于自然语言需求本身的模糊性和不准确性^[3]。Meincke 等人从自然语言处理的角度出发，对需求模糊性对代码生成质量的影响做了系统分析，认为提示的模糊性、非结构化和上下文依赖性都会使模型容易产生不符合用户需求的代码^[24]。Liu 等人对编程竞赛题做了大规模测评，证明未充分澄清的题目描述会导致不同大语言模型生成的代码功能正确性出现较大方差，且正确率随描述模糊度增加而下降^[25]。此外，关于代码安全性的研究表明，模糊需求下生成的代码更容易引入安全缺陷。Pearce 等人对 Copilot 生成代码的安全性进行审计后发现，约 40% 的生成代码存在 CWE-Top-25 级别的安全漏洞^[26]。因此，在代码生成的工作流程中加入需求澄清和轻量级安全扫描是有必要的。

本系统中 AnalyzerAgent 的模糊点识别和 QuestionAgent 的逐维度提问，是一种在生成前主动消解歧义的工程方案；系统在代码生成后自动检测硬编码凭据、eval()调用等危险模式，是对上述安全性研究的直接回应。通过“前置澄清+后置扫描”的双重机制，本系统旨在从源头降低模糊性带来的生成偏差，并在输出端提供一道安全防线。

2.2 交互式需求澄清技术

2.2.1 需求工程中的模糊性管理

软件需求中的模糊性是一个经典问题。Kamsties 等人对自然语言需求规格中的歧义类型做了系统分析，包括词法歧义、句法歧义、语义歧义、语用歧义^[27]。Femmer 等人进一步利用机器学习技术自动分类需求文档中的“需求异味”，包括模糊性、主观性和不完整性^[28]。近年来，Amna 和 Poels 提出的 AmbiTRUS 框架从

词汇、句法、语义和语用四个语言学层面构建了识别用户故事中潜在模糊性的系统化方法^[29]。这些基础研究为本系统将需求模糊点维度化提供了理论依据。系统把模糊点拆解成“输入规格”“输出规格”“边界条件”等维度，将语义和语用层面的歧义转化为可操作的分解。

在需求获取阶段，可以用知识图谱、本体的方法来降低模糊性。Dobson 和 Sawyer 认为结构化地提取“实体-属性-约束”是降低需求模糊性的关键^[30]。Aranda 等人则发现利用领域本体指导需求访谈能显著提升需求的完备性和一致性^[31]。本系统的技能管理模块可以创建出包括维度名称、引导问题、默认答案在内的澄清模板，把领域内的知识以结构化的形式添加到系统当中，模糊性的识别不再是零基础的，而是根据用户自己设置的知识来进行扩展，从而加快了模糊性的识别速度。

2.2.2 基于对话的交互式需求获取

交互式需求获取在对话式 AI 中有丰富先例。查询扩展技术通过同义词、概念图谱或用户交互日志扩充原始查询，以缓解模糊查询的低召回率问题^[32]。在代码领域，Specify-by-Example 方法让用户提供具体的输入-输出示例来明确需求，降低了描述负担^[33]。在对话式推荐系统中，Christakopoulou 等人提出基于主动学习的对话策略，通过选择信息增益最大的问题来快速收敛用户偏好^[34]。这些思想同样适用于代码需求的澄清：系统应能评估当前不确定性，生成高信息量的问题来高效获取用户意图。

更具体的，在智能代码助手中，Ross 等人提出了程序合成场景下的对话式澄清框架，将用户意图建模为多个可能程序，通过提出区分性测试用例来逐次缩小假设空间^[35]。本系统的 QuestionAgent 为每个模糊维度生成引导性问题，正是采用类似思路，通过多轮问答将模糊需求逐步精化为具体规格。

2.2.3 对话式澄清中的问题生成策略

在多轮澄清中，如何生成有效的问题是核心挑战。基于信息增益的提问策略就是选择可以最大程度上减少不确定性的问题^[36]。对话状态跟踪中的槽填充方法用指定的目标槽位驱动系统向用户提问缺失的槽值^[37]。本系统采用基于大语言模

型的问题生成方案：当 QuestionAgent 收到每一个模糊维度的描述之后，就会用 LLM 生成带选项示例的具体问题，并且系统会依次采集用户的回答来补充到 clarified_spec 字典中。通过逐个提问的方式进行澄清，保证澄清的过程是系统的，不会造成过多的信息重复或者遗漏。

为避免用户疲劳，系统还应控制提问的数量与质量。Rastogi 等人指出，单次会话的提问数量应有所限制，问题应尽可能简洁并提供预设选项^[38]。本系统的前端模糊点编辑器可以删除或者合并维度，主动控制澄清的粒度和数量，符合该设计建议。

2.2.4 结构化需求澄清框架：ChatCoder 与 ClarifyGPT

在 LLM 代码生成领域，两个代表性框架为交互式需求澄清提供了系统性的方法论。

ChatCoder^[39]设计了一个两轮对话框架——“释义与扩展”和“深入与回溯”，系统性地引导用户完善需求描述，如图 2.1 所示。

第一轮要求 LLM 从关键概念、方法目的、输入要求、输出要求、边缘情况、异常与错误六个角度扩展需求，用户审查并修正 LLM 的理解；第二轮则由 LLM 提出最关心的问题并生成猜测答案，用户逐一确认或修正，并可在发现理解错误时回溯至之前的问题点。实验表明，ChatCoder 将 GPT-4 在 Sanitized-MBPP 和 HumanEval 上的 Pass@1 分别从 66.15%和 81.10%提升至 76.65%和 90.24%，同时证明了人工审查在流程中不可或缺。本系统的交互流程直接借鉴了 ChatCoder 的“扩展-审查-提问-回溯”思想：用户首先编辑和确认模糊点列表（类似于审查 LLM 的理解），然后逐维度回答系统生成的问题（类似于深入提问），并可通过回滚按钮回溯修正（类似于回溯功能）。

ClarifyGPT^[40]是由 Mu 等人提出的通过需求澄清增强代码生成的框架，核心创新在于引入了“代码一致性检查”作为需求模糊性的自动判定依据。ClarifyGPT 包含四个主要阶段，如图 2.2 所示。

ClarifyGPT 方法分为四个步骤，首先根据大语言模型生成种子输入，然后进行基于类型的变异操作来生成测试输入；其次，生成多个不同的代码实现，在相同的测试输入上比较它们的输出是否一致，如果输出不一致，则判定需求存在模

糊性；第三步，将行为不一致的代码解作为示例，引导大语言模型分析模糊点并生成相应的澄清问题；最后，将用户回答与原始需求整合，生成增强后的最终代码。

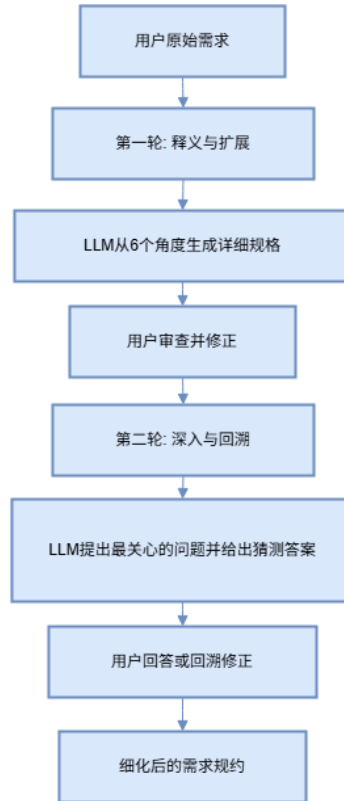


图 2.1 ChatCoder 对话框架流程图

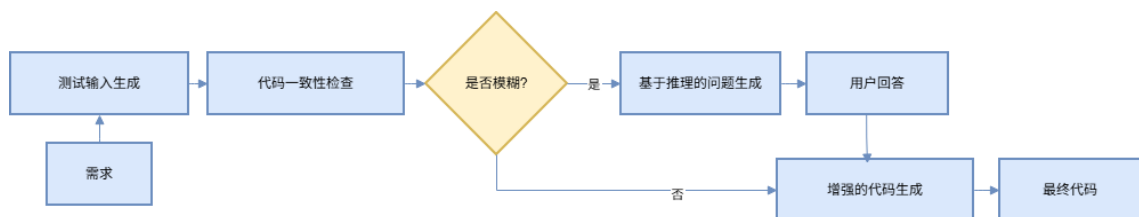


图 2.2 ClarifyGPT 对话框架流程图

该框架的主要思想就是使用不同的代码在同样的测试输入上输出不同的来检测需求的模糊性，从实验结果来看，在 MBPP-sanitized 上 ClarifyGPT 得到的相对提高为 13.87%，模糊性检测的准确率为 88.57%。本系统在生成实验组代码的同时会生成无澄清的对照组代码，用“一键测试”来比较两组代码的通过率，实际上就是一种轻量级的“一致性检查”，如果澄清后通过率明显高于未澄清组，则说明消解了原始歧义，反之则说明存在未被处理的模糊点。

2.2.5 历史经验学习与案例推理

随着系统的持续使用，历史成功案例成为提升生成质量的重要资源。基于案例的推理（Case-Based Reasoning, CBR）通过检索历史中与当前问题最相似的案例，将其解决方案复用或适配于新情境^[41]。在代码生成中，CEFC 框架从案例库中检索相似代码片段并在结构化知识指导下改写，显著提高了生成质量^[42]。TraceCoder 更进一步，不仅存储成功案例，还记录失败轨迹，通过回滚至历史最佳状态进行重试，增强了生成稳定性^[43]。

本系统 HistoryAgent 按照词重叠相似度在 hllm_memory.json 里找寻到最相似的历史案例，把它当作 few-shot 示例融入 CodeAgent 的生成提示之中，这就是 CBR 思想在代码生成提示工程里的直接运用。rollbackagent 存放了最近十个状态快照，用户可以随时回退到先前阶段，实现类似 TraceCoder 的探索回溯。

2.2.6 交互式需求澄清中的可解释性与透明性

在交互式智能系统中，可解释性对于建立用户信任至关重要。Meincke 等人的研究指出，提示的复杂性和条件性使得用户难以预测模型行为，因此系统应当通过结构化的交互界面降低这种不确定性^[2]。Wu 等人提出的 AI 链概念展示了通过将大语言模型提示链接起来，实现透明且可控的人机交互^[8]。这些研究为本系统的交互设计提供了重要指导：模糊点编辑界面的可视化表格、实验组与对照组代码的并排展示、代码解释与安全建议的自动生成，都是对可解释性和透明性原则的具体实现。

2.3 自动化测试与代码质量评估

2.3.1 基于 LLM 的自动化测试生成

代码生成后的正确性验证是确保系统输出可用的关键环节。传统测试生成主要依赖基于搜索的软件测试技术，如 EvoSuite 和 Randoop，它们通过遗传算法或随机方法生成测试输入以最大化覆盖率。近年来，LLM 开始在这一领域崭露头角。TestPilot 展示了利用 LLM 为 Java 方法自动生成 JUnit 测试的可行性，并通过编译

和运行结果筛选低质量测试^[44]。CodaMosa 进一步将传统基于搜索的方法与 LLM 相融合，在覆盖率停滞时调用 LLM 生成定向测试，成功突破覆盖率瓶颈^[45]。TitanFuzz 首次将 LLM 应用于深度学习编译器测试，展现了模型在复杂测试构造上的潜力^[46]。

本系统实现的“一键测试”功能就是使用 LLM 根据需求自动生成 pytest 测试用例，然后在临时沙箱目录中运行测试，解析 pytest 输出来计算通过率。该种“生成-测试-评分”闭环借鉴了以上工作主要思想，但是又用超时控制和结果解析的方式实现了工程上轻量化落地。

2.3.2 安全沙箱与代码执行隔离

由于生成的代码可能包含危险操作，测试执行必须进行严格的环境隔离。常用技术包括：基于 Linux namespace 的轻量进程隔离、Docker 容器化、WebAssembly 虚拟机及系统调用过滤等。OpenAI 在 Codex 评测环境中采用了基于 namespace 的沙箱^[21]。本系统为简化部署，采用 Python 的 `tempfile.TemporaryDirectory` 创建临时工作目录，所有文件读写和代码执行均限于此目录，并设置 30 秒超时以防止死循环耗尽资源。这虽然不如容器化隔离彻底，但对本系统面向的个人与小团队使用场景已提供了足够的安全保障。

2.3.3 代码质量的多维度评估

代码质量除功能正确性外，还包括可读性、可维护性、效率、安全性等多个方面。在自动化评估方面，传统静态分析工具能够检测部分代码异味，但在语义层面存在不足。LLM 则能够从更高级的视角进行评价。Pezzè 等人对 ChatGPT 生成代码的系统评估发现，LLM 在一定程度上可识别代码可读性问题和潜在缺陷，但与人类专家评估的一致性仍待提升^[25]。在安全性方面，Pearce 等人的审计工作^[26]迫使社区正视 LLM 生成代码中的安全漏洞风险。

本系统采用“自动评分+人工修正”的双阶段评估机制：先用 LLM 对代码进行 0-100 分自动评分并给出改进建议，随后用户可调整最终评分，评分结果持久化于 CodeRating 表中。此策略兼顾了自动评估的效率与人类专家判断的准确性，且

所收集的用户评分数据为后续基于偏好的模型对齐（如 DPO）提供了标注数据储备。

2.4 多智能体协作与人在回路

2.4.1 多智能体分工协作架构

随着 LLM 能力的增强，多智能体协作已成为解决复杂任务的有效范式。ChatDev 将瀑布式软件开发流程模拟为一个包含 CEO、CTO、程序员、测试员等角色的聊天室，通过自然语言对话完成从需求分析到代码交付的完整生命周期^[48]。MetaGPT 通过将软件工程专家的“标准操作流程”以结构化提示注入各个智能体，实现了需求分析、设计、编码、测试的端到端自动化。Self-Collaboration 则让同一模型扮演分析师、编码者和测试者三种角色进行自我对弈，在代码生成任务上取得优于单一生成的效果^[50]。这些工作共同验证了“分而治之”的多智能体范式的有效性。

Liu 等人对基于 LLM 的软件工程智能体进行了全面的综述，对目前多智能体在需求分析、代码生成、测试调试、项目管理等场景中的应用模式以及主要的挑战进行了梳理，认为智能体之间的有效协作和信息共享是提高整体效能的关键因素^[25]。Huang 等人提出的知识引导多智能体协作框架，也表明了把领域知识注入到多个智能体中，从而提高需求工程智能化水平的可行性^[23]。借此，本系统的核心业务逻辑采用的设计是 Orchestrator 作为中央协调器，负责状态转移和调用各个专责智能体；AnalyzerAgent 负责模糊点识别；QuestionAgent 负责生成引导性问题；CodeAgent 负责代码生成；HistoryAgent 管理历史案例；RollbackAgent 处理状态回溯。各 Agent 的底层都是 LLM 的，但是采用了不同的提示模板和功能划分，从而使得各个模块可以独立地进行改进。

2.4.2 人在回路的设计原则与交互准则

“人在回路”（Human-in-the-Loop, HITL）理念认为在 AI 系统决策的关键节点上保留人的参与。Amershi 等人对 200 多篇人机交互研究进行分析，提出了 AI 系统 18 条设计指南，其中系统应该显示决策置信度、允许用户反馈、支持高效校

正、覆盖^[51]。Clemmensen 等人从社会技术视角出发，构建了 HITL 系统设计的研究框架，提炼出透明性、可控性和互补性三项核心原则^[52]。在具体的交互策略上，Mozannar 等人发现，选择合适的求助时机能显著提升用户对系统的信任感与协作效率^[53]。

本系统交互设计符合上述原则，模糊点编辑阶段用户可以增删改系统推荐的维度，具有可控性；实验组和对照组代码并排展示并附测试通过率，具有透明性；自动评分和用户评分并存，具有互补性。系统使用状态机控制各维度提问的节奏，防止频繁打断用户工作。

2.4.3 反馈驱动的对齐与持续优化

人在回路的深层价值在于利用人类反馈持续优化模型。利用人类反馈优化模型是当前的研究方向。本系统通过收集用户评分和高质量案例标记，为后续基于偏好学习的模型优化积累了标注数据。

2.5 对话状态管理与交互策略

2.5.1 任务型对话系统的形式化

任务型对话系统旨在通过多轮人机对话帮助用户完成特定任务。POMDP 框架为对话管理提供了数学基础，通过信念跟踪在不确定性下进行最优行动选择^[54]。基于有限状态机和槽填充的方法在工业界更为常见，因其行为可预测且易于工程实现。本系统的 Orchestrator 本质上是一个槽填充式的对话管理器：clarified_spec 字典即为槽值集合，每个模糊维度被视为一个待填充的槽，问题索引依次推进直至所有槽值填充完毕，然后触发代码生成。这种 FSM 设计保证了对话流程的可控性和可解释性，非常适合于本系统这种槽位数量不大且相对固定的澄清场景。

2.5.2 对话状态的持久化与恢复

在多轮对话 Web 应用中，会话上下文的持久化是关键工程问题。常见方案包括服务端会话存储（如 Redis）、客户端存储（localStorage/sessionStorage）以及令牌化对话状态。Fielding 在其博士论文中提出的 REST 架构风格对 Web 应用的状态

管理产生了深远影响，其无状态约束要求每个请求包含完成请求所需的所有信息^[55]。本系统采用前后端协同的持久化策略：前端利用 `sessionStorage` 暂存消息和代码，后端在每个状态转移后通过 `orchestrator_state` 字段将完整快照持久化至 MySQL。这既保证了页面刷新时的快速恢复，又保障了长期可靠的状态存档。

2.5.3 交互策略的用户适应性

不同用户对交互频率和澄清深度的偏好存在差异。Schubert 等人的研究发现，经验丰富的开发者倾向于少量高信息量的提问，而新手则更接受较多引导性问题^[56]。本系统的技能模板机制和模糊点手工编辑功能允许用户根据自己的经验水平定制澄清粒度，从而在一定程度上适应了用户差异性。系统默认的六维度模糊检查清单则为新手提供了结构化的引导框架，降低了使用门槛。

2.6 系统架构与工程实现

2.6.1 FastAPI 框架与 RESTful API

FastAPI 是现代高性能的 Python Web 框架，可以用来创建 API。它底层使用的是 Starlette（提供异步路由、中间件的支持），Pydantic（提供数据校验、序列化），原生支持 Python 的 `async/await` 异步 I/O 机制^[57]。FastAPI 对于大量的 I/O 密集型并发请求（数据库查询、调用外部 API 等）来说，它的资源使用情况比传统的同步框架 Flask 要好多。本系统后端使用 FastAPI 构建，主要考虑两个优势，一是原生异步支持，适合需要频繁调用外部 LLM 服务的高并发场景；二是自动生成 OpenAPI 文档，方便前后端联调。使用依赖注入的方式可以很好地解决认证授权 `get_current_user`，数据库会话 `get_db` 等横切关注点的问题。API 使用的是 RESTful 风格，资源的语义清楚，利于前后端分离开发。

2.6.2 数据库设计与多用户数据隔离

SQLAlchemy 是 Python 领域最成熟的对象关系映射（ORM）框架。它给开发者提供了一个高层次的 ORM API 以及底层的 Core API，使用 Python 类和对象就

可以操作数据库，自动处理 SQL 语句的生成、连接池的管理、事务的控制。本系统使用了 SQLAlchemy ORM 来对 MySQL 数据库进行管理。MySQL 是常用的一种数据库，具有事务 ACID 特性、索引优化、稳定的主从复制等特点，适合用来存储用户信息、评测任务记录等主要业务数据。多用户数据隔离就是通过所有的查询中根据 `current_user.id` 进行行级过滤来实现的，这是多租户 SaaS 应用的标准做法，保证每个用户只能访问自己的数据^[58]。此外，`orchestrator_state` 字段采用 JSON 格式弹性存储非结构化的工作流状态，既利用 MySQL 的 JSON 函数实现部分查询能力，又避免了为每种状态单独建表的复杂性，兼顾了性能与灵活性。

2.6.3 身份认证与会话安全

JSON Web Token (JWT) 是一个开放标准 (RFC 7519)，它用数字签名过的紧凑格式来把用户的标识以及过期时间这样的声明表达出来。JWT 认证不同于传统的基于会话的存储 Session，JWT 认证是无状态的，服务器不需要维护会话存储，便于横向扩展。本系统使用 JWT 无状态认证^[59]，用户登录后获得一个由 HMAC-SHA256 算法签名的令牌（令牌中不包含敏感信息），前端在后续请求的 Authorization 头部中携带该令牌。用户密码安全方面，系统使用 PBKDF2-SHA256 密钥派生算法，进行 10 万次迭代哈希并加盐后存储^[60]。PBKDF2 是一种自适应哈希函数，通过重复计算和盐值随机化可以有效地抵抗离线暴力破解和彩虹表攻击。前端使用 AdminRoute 实现基于角色的路由守卫，只允许具有相应权限的用户访问管理界面；后端在每个 API 入口处再次校验令牌和权限，双重保障。

2.6.4 前端交互与可视化

React 是由 Meta 维护的一个前端库，使用函数式组件、Hooks 声明式编程模式、虚拟 DOM 来高效地完成视图的更新。本系统前端使用的是 React 的函数式组件，使用的是 Ant Design 组件库。Ant Design 是一套企业级的 UI 设计语言以及 React 实现，提供了表格、表单、模态框等开箱即用的组件，可以快速搭建出风格统一的界面。使用 react-router-dom 完成单页应用(SPA)的路由切换，防止由于路由切换而造成页面刷新，类似于原生应用的交互方式。代码展示模块使用 react-syntax-highlighter 来对各种编程语言进行语法高亮，并且可以支持各种主题样

式，使 LLM 输出的代码块更加容易阅读。数据分析页面使用 recharts 图表库画出了评分趋势折线图以及模型对比柱状图。Recharts 是一个基于 React、D3 声明式地组合图表组件，具有响应式的布局。前端技术的选择是 web 应用成熟度的表现，也是本系统给用户提供的信息量大、流畅的用户界面。

2.7 本章小结

本章围绕“基于交互式需求澄清的智能代码生成系统”的设计与实现，从大语言模型代码生成、交互式需求澄清、自动化测试与评估、多智能体协作与人在回路、对话状态管理与交互策略、系统架构与工程实现六个方面系统梳理了相关理论技术。首先阐述了 LLM 代码生成的能力演进及模糊需求下暴露的局限性，明确了引入澄清机制的必要性。随后深入分析了需求工程中的模糊性管理、对话式需求获取、结构化澄清框架（ChatCoder、ClarifyGPT）、案例推理与回滚、可解释性与透明性等核心技术，为系统的交互流程与 Orchestrator 设计提供了直接依据。接着论述了自动测试生成、安全执行与代码评估等方法，展示了生成-测试-反馈闭环的工程依据。然后剖析了多智能体分工与 HITL 交互原则，验证了系统架构的合理性和人本导向。最后从 Web 框架、数据库隔离、认证安全与前端可视化等角度夯实了技术选型基础。上述技术共同构成了本系统的理论支撑体系，为后续章节的需求分析、系统设计及实验评估提供了坚实的理论与实践参考。

第3章 系统需求分析

3.1 需求分析

随着软件行业的发展，各种应用系统越来越大、越来越复杂，软件开发也从原来的“编写代码”向“工程化协作”转变。但是实际开发活动当中，由于开发人员、项目经理以及最终用户对于需求的表述不清、理解有误而导致反复返工、延期甚至项目失败的现象时有发生。对初学者、独立开发者来说，将模糊的思想变成具体的、可行的代码规格是比较困难的。

通过对个人编程实践及周边开发者群体的观察，当前需求分析与代码生成环节主要存在以下三个典型问题：

1) 需求表述不清，没有体系化的澄清方式。用户提出的大部分需求都是高度概括的，比如做一个贪吃蛇游戏、写一个合并排序函数等等，对于输入类型、边界情况、异常处理、性能要求等主要问题没有涉及。传统模式下是依靠人工反复的交流，耗时费力，并且容易忽视重要的一面。

2) 沟通成本大，开发者和用户之间存在着认识上的差异。因为双方知识背景不同，用户对技术实现概念的把握不大，开发者对业务场景的了解也不够深入，所以澄清对话效率低，重大的决策点很容易被忽略。缺乏系统的引导，常常出现你以为说清楚了，但实际没有说清楚的情况。

3) 从需求到代码的转换依靠个人能力，质量参差不齐。即使需求已经基本确定，不同水平的开发者产出的代码在健壮性、可读性、安全性、性能上也存在着很大的差异。初学者容易写出生死存亡都取决于输入条件的代码，或者不考虑安全性的代码。市场上虽然已经出现了许多通用大模型辅助编程工具，但是大多数都需要用户直接给出非常精确的提示词，没有解决“先想清楚需求”的核心前提。

为了解决以上问题，本文设计并实现了一个交互式的代码生成系统，该系统可以对需求进行交互式的澄清。系统把需求分析和代码生成结合起来，用户输入原始需求之后，系统会自动识别出其中的模糊点，用结构化的问卷方式逐步引导用户补充细节，直到形成完整、无歧义的规格说明；之后调用大语言模型根据规格说明生成符合要求的代码，并自动检测潜在的安全风险（SQL 注入、硬编码秘

钥等）并给出建议。流程使代码质量得以提高，同时需求分析的门槛也降低了，非专业的用户也能很快地把想法变成运行的程序。

该系统对于促进软件工程教育、推动低代码开发、降低开发者入门门槛都有积极的实践意义，也给基于大模型的需求工程工具探索了一条可行之路。

3.2 功能性需求分析

3.2.1 业务用例分析

通过调研和分析，确定系统的参与者包括普通用户和管理员两大角色。管理员是普通用户的特化，除自身业务外还继承普通用户的全部功能。

1) 普通用户业务用例

普通用户是系统的最终服务对象，其主要业务活动包括：账户管理、需求澄清与代码生成、代码分析、评分代码、标记高质量案例、查看历史任务、查看规格说明书、技能管理以及系统首页仪表盘查看。普通用户的业务用例图如图 3.1 所示。

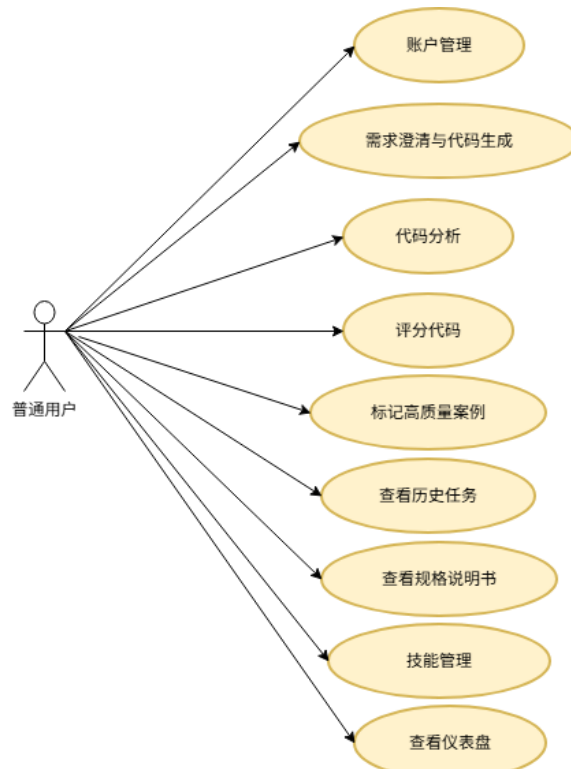


图 3.1 普通用户业务用例图

（2.）管理员业务用例

管理员除继承普通用户的全部业务外，还拥有系统管理相关的业务，包括：用户管理（增加、编辑、删除用户、重置密码）、模型配置管理（增加、修改、删除模型配置、启停模型）以及代码分析（评分趋势、模型对比、测试通过率统计）。管理员业务用例图如图 3.2 所示。

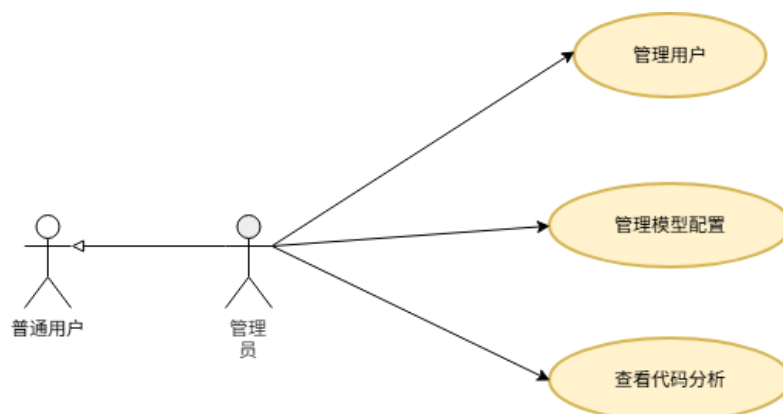


图 3.2 管理员业务用例图

3.2.2 系统用例分析

为了清晰表达本系统的功能性需求，本节采用 UML 用例图对系统行为进行建模。用例分析从三类场景出发：核心业务（需求澄清与代码生成）、代码评价与案例管理、系统管理功能。

1) 需求澄清与代码生成

输入需求，识别模糊点并生成澄清问题，用户回复后调用大模型生成代码并保存规格文档，最后轻量化测试。如图 3.3 所示。

2) 代码评价与案例管理

支持自动/手动评分代码，标记高质量案例入库，可查看、下载或删除案例库中的代码。如图 3.4 所示。

3) 系统管理功能

管理员可增删改用户、重置密码；也可新增、编辑或删除模型配置。如图 3.5 所示。

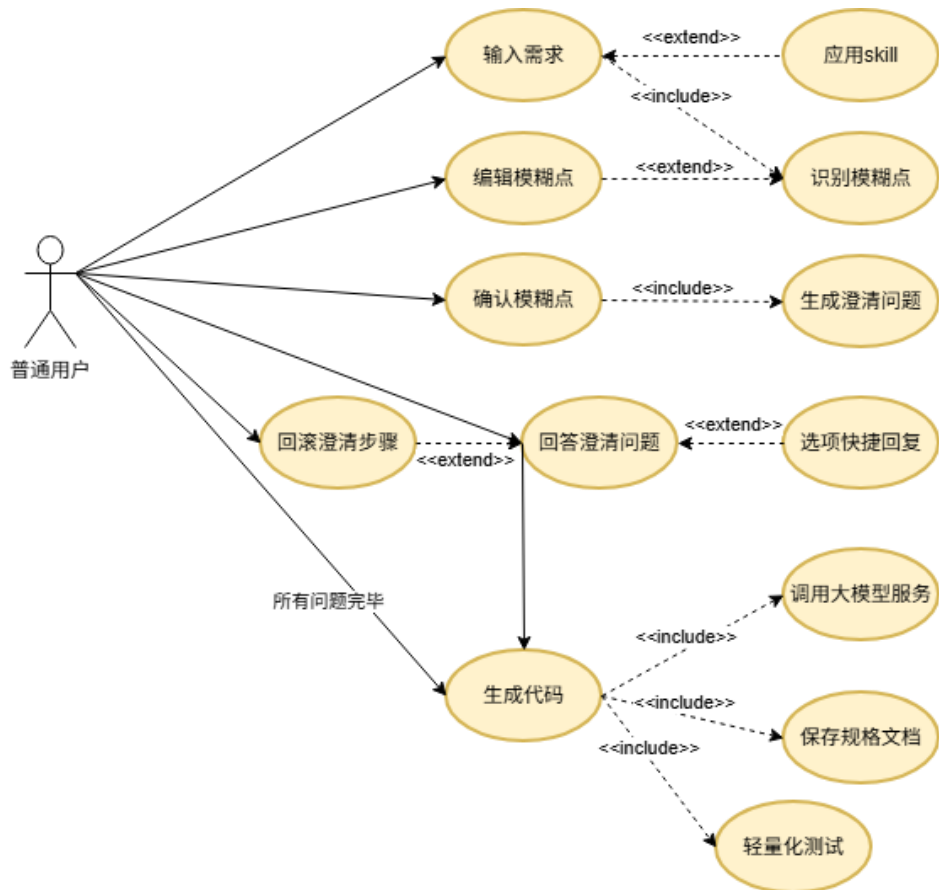


图 3.3 需求澄清与代码生成用例图

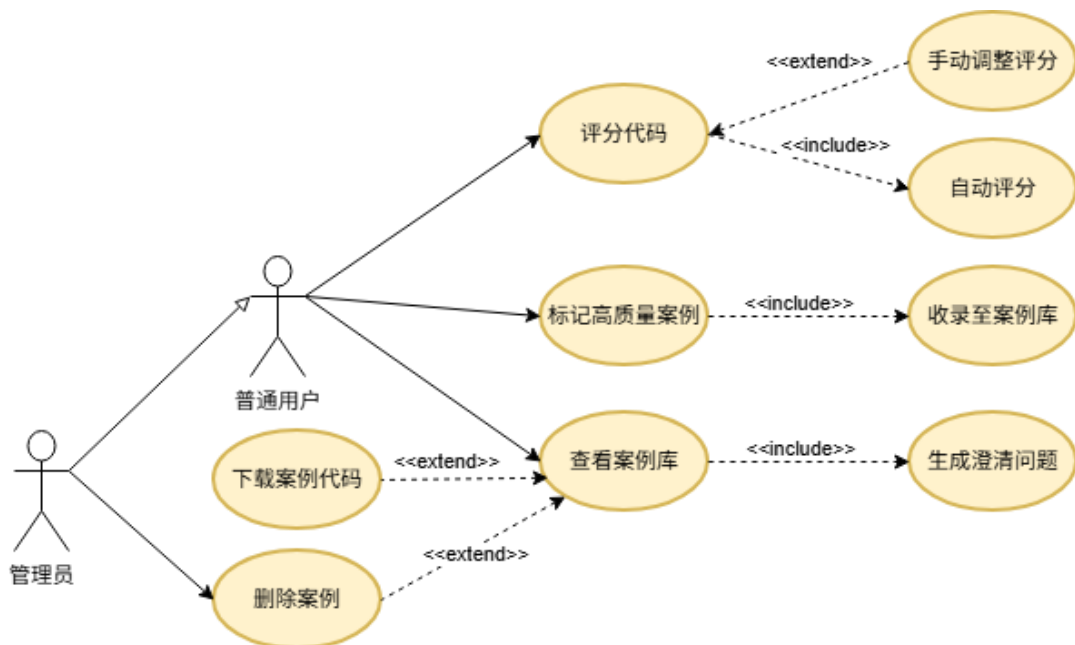


图 3.4 代码评价与案例管理用例图

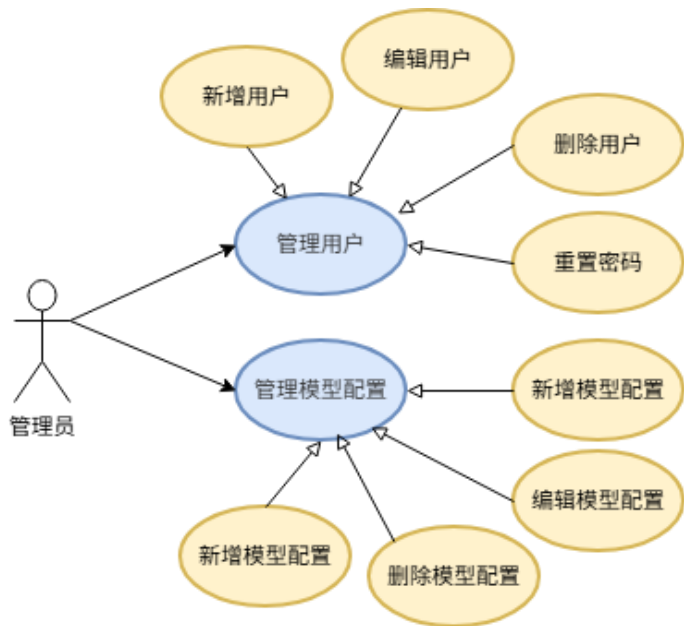


图 3.5 系统管理功能用例图

3.2.3 用例规约

用户注册用例规约表如表 3.1 所示。

表 3.1 用户注册用例规约

用例描述	
用例名称	用户注册
标识符	UC-REG
参与者	普通用户（未登录状态）
描述	用户填写用户名和密码注册账号，成功后自动登录并进入系统。
前置条件	用户未登录，位于注册页面。
后置条件	注册成功，生成 JWT 令牌，用户进入系统首页。
基本操作流程	1. 用户点击注册链接进入注册页面；
	2. 填写用户名（不小于 3 字符）；
	3. 填写密码（不小于 6 位）；
	4. 再次输入密码确认；
备选流程	5. 点击“注册”按钮；
	6. 系统校验两次密码是否一致，检查用户名是否已存在；
	7. 若通过，创建用户记录，密码加密存储，返回 JWT 令牌；
	8. 前端保存令牌，跳转至仪表盘。
假设	6a. 两次密码不一致：提示错误，返回步骤 3； 6b. 用户名已存在：提示错误，返回步骤 2。
假设	用户浏览器已连接网络，后端服务正常运行。

用户登录用例规约表如表 3.2 所示。

表 3.2 用户登录用例规约

用例描述	
用例名称	用户登录
标识符	UC-LOGIN
参与者	普通用户（未登录状态）
描述	用户输入用户名和密码，系统验证通过后颁发令牌，允许访问受保护资源。
前置条件	用户已注册，未登录。
后置条件	登录成功，获得 JWT 令牌，进入系统。
基本操作流程	1. 用户进入登录页面； 2. 输入用户名、密码； 3. 点击“登录”； 4. 系统校验用户名和密码； 5. 校验通过，创建访问令牌并返回； 6. 前端保存令牌，跳转至仪表盘。
备选流程	4a. 用户名或密码错误：提示“用户名或密码错误”，返回步骤 2。
假设	用户浏览器已连接网络。

需求澄清与代码生成用例规约表如表 3.3 所示。

表 3.3 需求澄清与代码生成用例规约

用例描述	
用例名称	需求澄清与代码生成
标识符	UC-CLARIFY-GEN
参与者	普通用户
描述	用户输入自然语言需求，系统识别模糊点并允许用户编辑；随后通过多轮问答逐步细化需求，最终由后台大模型生成符合要求的代码，并附有安全建议。
前置条件	用户已登录，已选择至少一个可用的大模型。
后置条件	生成代码并展示在界面，同时自动保存为规格文档；会话状态被持久化。
基本操作流程	1. 用户在“需求澄清助手”页面选择生成模型； 2. 输入需求描述（如“生成一个计算斐波那契数列的函数”）； 3. 系统分析需求，返回一组织识别出的模糊点，以列表形式展示，允许用户增删改； 4. 用户确认模糊点后，系统逐一提出澄清问题，可能以选项形式给出； 5. 用户回答当前问题，系统记录答案，然后提出下一个问题； 6. 所有问题回答完毕后，系统调用选定大模型生成代码； 7. 系统展示代码、必要解释以及安全建议（如检测到 SQL 拼接时提示注入风险）； 8. 自动保存规格文档（包含原始需求、澄清后的规格、对话历史等）。
备选流程	3a. 用户对模糊点不满意：可点击“新增”手动添加，或编辑、删除现有项； 5a. 用户希望回滚到上一问题：点击“回滚”按钮，恢复到上一状态并重新提问。 5b. 用户输入非法答案：系统提示重新输入； 7a. 代码生成失败或超时：系统提示错误信息，建议重试。
假设	大模型 API 服务可用，网络正常。

应用技能用例规约表如表 3.4 所示。

表 3.4 应用技能用例规约

用例描述	
用例名称	应用技能
标识符	UC-SKILL-APPLY
参与者	普通用户
描述	用户选择一个预定义的技能（含一组澄清模板），系统自动创建新会话并将这些模板作为预设的模糊点，跳过相应问题的提问。
前置条件	用户已登录，存在至少一个技能。
后置条件	新会话创建，技能模板被标记为已澄清，直接进入后续提问或代码生成。
基本流程	1. 进入“技能管理”页面； 2. 点击某技能的“应用”按钮； 3. 后端创建会话，加载技能关联的模板； 4. 前端跳转到需求澄清助手，并提示已应用技能。

一键测试用例规约表如表 3.5 所示。

表 3.5 一键测试用例规约

用例描述	
用例名称	一键测试
标识符	UC-TEST
参与者	普通用户
描述	用户对已生成的实验组和对照组代码执行自动化测试，系统生成测试代码并运行，返回通过率。
前置条件	实验组和对照组代码均已生成，且代码通过可测试性检查（无 GUI/Web 框架、包含函数定义）。
后置条件	测试结果保存到数据库，前端展示通过率对比。 1. 用户点击“一键测试”按钮； 2. 后端调用可测试性检查； 3. 根据原始需求生成测试代码； 4. 分别运行实验组和对照组代码的测试； 5. 计算通过率并保存； 6. 前端展示结果并提供跳转到分析页面的链接。
备选流程	2a. 代码不可测试（无函数/含 GUI）→ 提示具体原因，终止测试。

用户评分与案例标记用例规约表如表 3.6 所示。

表 3.6 用户评分与案例标记用例规约

用例描述	
用例名称	用户评分与案例标记
标识符	UC-RATE-MARK
参与者	普通用户
描述	代码生成后，用户可查看自动评分，并给出自己的评分；如果用户评分 ≥ 85 分，系统建议将此次生成结果标记为高质量案例，存入案例库供后续参考。
前置条件	已生成代码，用户处于代码展示页面。
后置条件	评分被记录，可选择将案例加入高质量案例库。
基本操作流程	1. 系统调用大模型对生成的代码进行自动评分，显示分数； 2. 用户进入评分弹窗，系统预填自动评分； 3. 用户可手动修改评分（0~100）； 4. 用户点击“提交评分”； 5. 系统存储评分记录； 6. 若用户评分≥ 85，弹出提示“是否收录到高质量案例库？”； 7. 用户确认后，系统将需求、澄清规格、代码等信息存入案例表。
备选流程	3a. 用户不修改评分，直接提交； 6a. 评分< 85，不触发案例标记提示，用户仍可在历史页手动点击“收录案例”。
假设	自动评分依赖大模型正常工作。

管理员模型配置管理用例规约表如表 3.7 所示。

表 3.7 管理员模型配置管理用例规约

用例描述	
用例名称	模型配置管理
标识符	UC-MODEL-ADMIN
参与者	管理员
描述	管理员可以在“模型配置”页面添加新的模型 API 配置，编辑现有模型的密钥或地址，启用/停用模型，以及删除模型。
前置条件	管理员已登录，进入模型配置页面。
后置条件	模型配置更新，系统中相应模型立即可用或不可用。
基本操作流程	1. 管理员点击“新增模型”； 2. 填写模型名称标识、API Key、可选 API Base URL，选择是否启用； 3. 点击保存，系统验证后存入数据库； 4. 管理员可通过开关直接启用/停用模型； 5. 编辑或删除模型时，点击对应按钮，填写信息或确认后执行。
备选流程	2a. 模型名称重复：提示错误，要求修改； 5a. 删除模型：系统弹出二次确认，防止误删。
假设	管理员具有相应权限。

3.2.4 系统用例活动图

为了更好地展示核心业务流程的动态行为，核心业务（需求澄清与代码生成）活动图如图 3.6 所示。

核心业务活动流程如下：用户输入需求后，系统判断是否存在模糊点。若有，则展示模糊点列表供用户编辑，系统生成澄清问题并由用户回答，记录答案后循环直至所有问题回答完毕。随后系统调用大模型生成代码，经测试后自动保存规格文档。

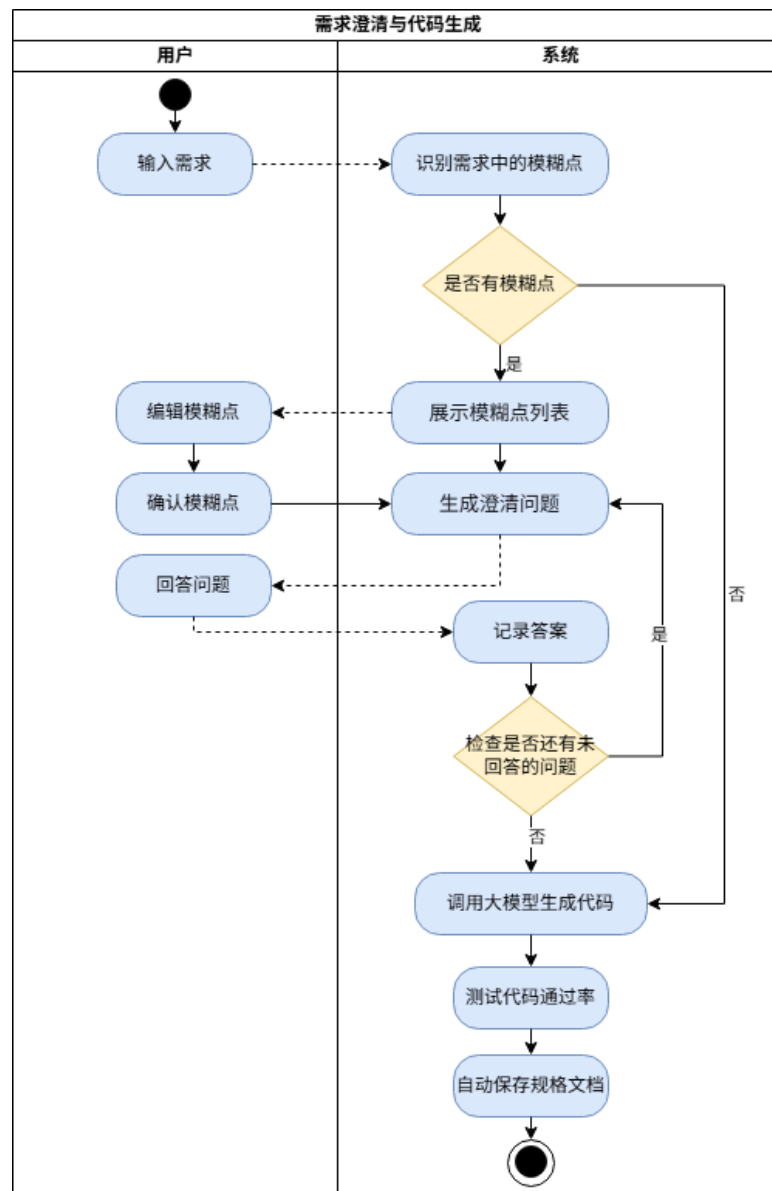


图 3.6 需求澄清与代码生成活动图

3.3 非功能性需求分析

3.3.1 性能需求

1.响应时间：前端页面首屏加载不应超过 2 秒；需求分析（模糊点识别）和代码生成接口的响应时间应在 10 秒内（视模型 API 延迟情况，通常应在 40 秒左右返回）；普通查询（如历史列表、案例列表）应在 2 秒内完成。

2.并发处理：系统应支持同时至少 50 个用户的正常使用，后端采用异步框架 FastAPI，可充分利用服务器资源，避免阻塞。

3.资源占用：前端构建产物应尽可能小，利用代码分割和懒加载；后端服务内存占用在空闲时应低于 200MB，正常使用时不超过 1GB。

3.3.2 安全需求

1.身份认证：所有敏感接口必须通过 JWT 令牌验证，令牌设置合理过期时间（默认 30 天），并在必要时可主动撤销。

2.密码安全：密码采用 PBKDF2-SHA256 算法加随机盐哈希存储，禁止明文传输或存储。

3.权限控制：普通用户不能访问管理员专属接口（如用户管理、模型配置管理），后端对所有管理接口强制检查 is_admin 字段。

4.数据隔离：普通用户只能查看自己的历史会话、评分和案例；通过用户 ID 进行数据过滤。

5.防注入：所有数据库操作使用 ORM 参数化查询，避免 SQL 注入；对大模型输入进行简单校验，但主要依赖 API 侧的安全措施。

6.敏感信息保护：API Key 等配置存储在环境变量或数据库，前端不暴露真实密钥。

3.3.3 可靠性需求

1.状态持久化：会话状态（澄清进度、已回答问题）实时保存至数据库，即使用户关闭浏览器或服务重启，重新登录后仍可恢复之前的对话。

2.异常处理：后端对所有可能的异常进行捕获，返回友好的错误信息，避免敏感信息泄露；对大模型 API 调用设置重试机制（最多 2 次），防止临时网络波动导致失败。

3.数据备份：数据库应定期备份，建议配置自动备份策略。

3.3.4 可扩展性需求

1.模型扩展：可通过后台管理界面直接添加新的大模型，无需修改代码；代码生成代理支持通过配置切换模型。

2.功能扩展：澄清模板、评分标准等均设计为动态配置，管理员可根据需要灵活调整，无需重新部署。

3.前端组件化：React 组件结构清晰，页面与业务逻辑分离，新增功能模块时只需增加新路由和页面组件即可。

3.4 约束需求

约束需求描述了系统设计和实现中的限制条件：

CR1：后端必须使用 Python 语言和 FastAPI 框架。

CR2：前端必须使用 React 框架，前后端通过 RESTful API 通信。

CR3：数据库必须使用 MySQL。

CR4：API 密钥通过环境变量管理，不得硬编码。

CR5：系统需支持至少 5 轮以上的连续对话，状态不丢失。

CR6：系统内置轻量化一键测试，只限于函数式代码。

3.5 本章小结

本章对智能代码生成系统进行了全面的需求分析。首先进行了总体的需求分析。然后采用面向对象方法，建立了用户和管理员的业务用例模型，并进一步细化了系统用例图，通过包含、扩展、泛化关系明确了用例间的联系。通过编写关键用例的详细规约表，明确了每个功能的参与者、前置后置条件、基本流程和备选流程。活动图直观展示了核心业务流程的动态逻辑。最后，从性能、安全、可

靠性、易用性、可扩展性以及设计约束六个方面提出了非功能性需求，为后续的架构设计、数据库设计和编码实现提供了明确的约束与目标。需求分析过程为系统的成功开发奠定了坚实基础。

第4章 系统概要设计

4.1 系统总体架构设计

本系统采用前后端分离的 B/S 架构模式，整体分为表现层、业务逻辑层、数据持久层和 AI 服务层四个层次。前端专注于用户交互与结果展示，后端负责核心业务逻辑编排与智能体协作。系统总体架构如图 4.1 所示。

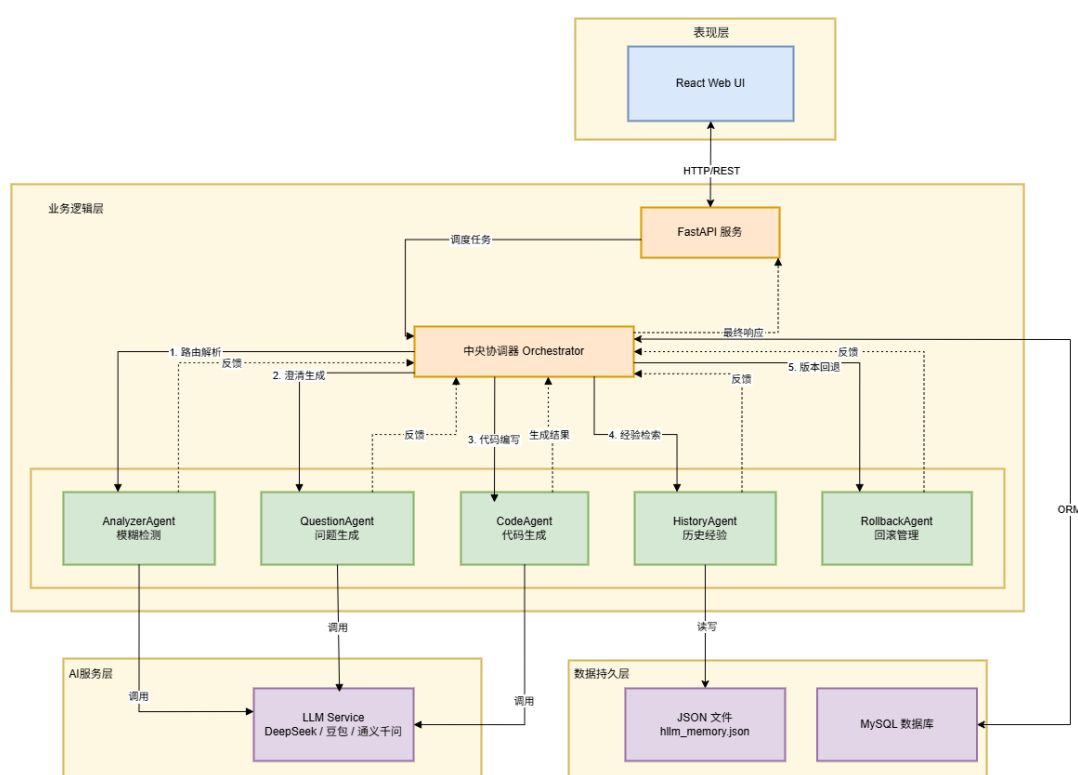


图 4.1 系统总体架构图

1.表现层（前端 Web UI）

前端基于 **React** 框架构建单页面应用，使用 **Ant Design** 组件库实现统一的界面风格。该层负责提供需求输入与对话交互界面、代码预览与操作界面、用户认证与权限管理界面，以及与后端 **API** 的数据通信。前端通过 **HTTP** 代理将 **API** 请求转发至后端服务，实现前后端分离部署。

2.业务逻辑层（后端 FastAPI 服务）

业务逻辑层采用 Python 语言的 FastAPI 框架构建，提供 RESTful 风格的 API 接口。该层的核心是中央协调器（Orchestrator），它采用基于有限状态机的多智能体协作架构，调度五个专业化智能体协同工作：

- 1) 需求分析智能体（AnalyzerAgent）：负责检测需求中的模糊点。
- 2) 问题生成智能体（QuestionAgent）：基于模糊点生成澄清问题。
- 3) 代码生成智能体（CodeAgent）：基于澄清后的规约生成代码和解释。
- 4) 历史经验智能体（HistoryAgent）：实现历史经验学习机制（HLLM），记录成功案例并支持相似检索。
- 5) 回滚管理智能体（RollbackAgent）：实现回滚机制，维护状态栈，处理冲突。

3.AI 服务层（LLM Service）

AI 服务层通过统一的客户端接口封装对不同大语言模型的调用。系统通过数据库中的模型配置表动态管理多个 LLM 提供商（如 DeepSeek、豆包、通义千问等），支持运行时切换模型。LLM 服务作为核心推理引擎，为需求分析、问题生成、代码生成等环节提供语言理解与生成能力。

4.数据持久层（Database）

数据持久层采用 MySQL 关系型数据库进行数据存储，通过 SQLAlchemy ORM 框架实现对象关系映射。主要数据实体包括：用户信息表（users）、会话记录表（sessions）、模型配置表（model_configs）、澄清模板表（clarification_templates）、高质量案例表（high_quality_cases）、代码评分表（code_ratings）和需求规格文档表（spec_documents）。此外，历史经验数据以 JSON 格式持久化存储在文件系统中。

4.2 系统功能模块设计

根据需求分析阶段的成果，将系统功能划分为用户认证模块、需求澄清模块、代码生成模块、历史经验模块、案例管理模块和系统管理模块六大功能模块。系统功能模块划分如图 4.2 所示。

各功能模块的详细描述如表 4.1 所示。系统从用户认证、需求澄清、代码生成、历史经验、案例管理及系统管理六个维度划分核心业务，覆盖从身份验证到需求

细化、代码输出、经验复用及后台运维的完整链路，为后续的设计与实现提供清晰的功能边界。

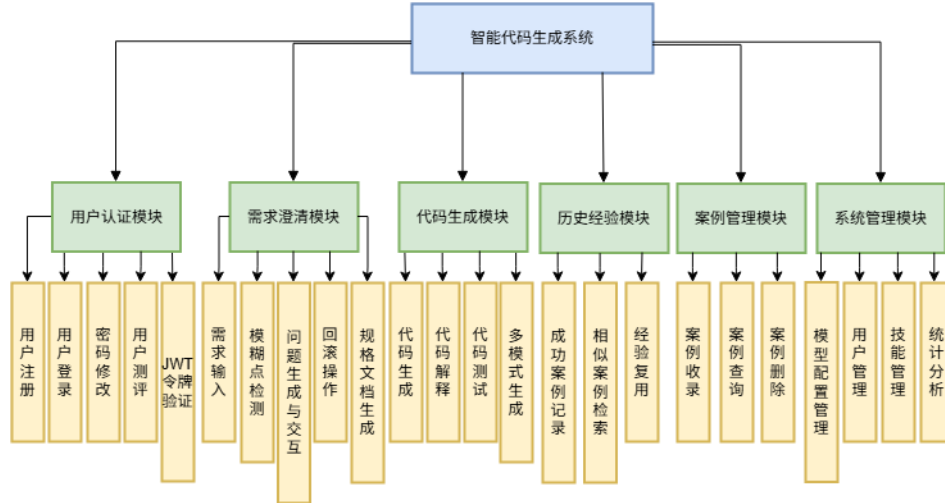


图 4.2 系统功能模块图

表 4.1 系统功能模块描述表

业务模块	功能划分	简要描述
用户认证模块	用户注册	新用户创建账号并设置密码
	用户登录	验证用户身份并颁发访问令牌
	密码修改	用户修改个人登录密码
	用户测评	测评设置用户质量分权重
	令牌验证	基于 JWT 的请求身份校验
需求澄清模块	需求输入	用户输入自然语言形式的功能需求
	模糊点检测	自动识别需求中的缺失信息和模糊表述
	问题生成与交互	生成引导性澄清问题并进行多轮对话
	回滚操作	支持用户修改已确认的答案
代码生成模块	规格文档生成	生成结构化的需求规约文档
	代码生成	生成澄清前后的代码
	代码解释	提供代码的自然语言解释
	代码测试	测试代码通过率
历史经验模块	多模式生成	支持 deepseek、doubao、qwen 三种模式
	成功案例记录	持久化存储成功完成的澄清-代码对
	相似案例检索	基于关键词匹配查找历史相似案例
案例管理模块	经验复用	将历史规约作为参考提示注入对话
	案例收录	将高质量代码标记收录到案例库
	案例查询	支持查看和检索已收录案例
系统管理模块	案例删除	管理员可删除不合格案例
	模型配置	管理可用的 LLM 提供商及其 API 密钥
	用户管理	管理员对系统用户进行增删改查
	技能管理	用户可创建技能并应用会话
	统计分析	用户会话统计与代码质量评分分析

4.3 系统工作流程

本系统围绕“输入需求→检测模糊点→交互澄清→生成代码”的核心流程展开，各角色在系统中的操作流程如图 4.3 所示。

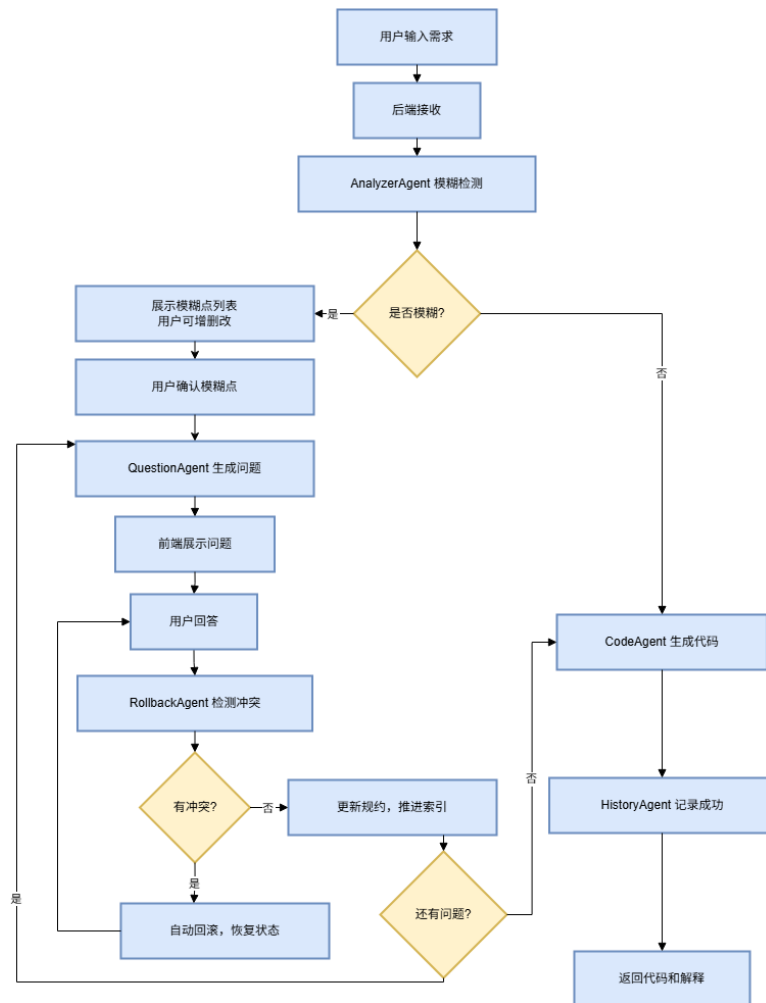


图 4.3 系统工作流程图

1. 用户输入初始需求。用户通过前端对话界面，以自然语言形式描述期望实现的功能。系统为该用户创建唯一会话标识，并将需求文本提交至后端。

2.模糊点检测。后端接收需求后，需求分析智能体（AnalyzerAgent）采用“开放式反思+结构化核对”的两阶段提示策略对需求进行分析。同时，历史经验智能体（HistoryAgent）基于关键词重叠度检索历史相似成功案例，作为参考上下文。

3.判断需求清晰度。若检测结果显示需求描述充分且不存在模糊点，则直接跳转至步骤 8 进入代码生成环节。

4.展示模糊点并允许编辑。若检测到模糊点，系统将模糊点列表返回前端。用户可以查看、增加、修改或删除模糊点，确认后进入澄清流程。

5.生成并推送澄清问题。问题生成智能体（QuestionAgent）为每个模糊点生成一个具体的引导性问题，系统按顺序逐个推送给用户。

6.用户回答问题。用户输入答案后，系统首先检测该回答是否与已有规约产生冲突。若冲突，回滚管理智能体（RollbackAgent）自动将对话状态恢复至上一个状态并重新提问；若无冲突，则正常整合答案。

7.判断澄清是否完成。系统检查是否所有模糊点均已澄清。若还有未回答的问题，返回步骤 5 继续推送；若全部完成，则将对话状态切换为“已确认”。

8.代码生成。代码生成智能体（CodeAgent）读取澄清后的完整需求规约，构造包含原始需求与澄清细节的提示词，调用 LLM 生成澄清前后的代码和解释。若存在相似历史案例代码，则作为参考注入提示词。

9.代码测试。内置轻量化一键测试，可直接测试函数代码通过率。

10.经验记录与结果返回。历史经验智能体将本次成功的需求-规约-代码三元组存入经验库。生成的代码和解释返回前端展示，同时自动保存需求规格文档。

4.4 对话管理状态机设计

为了规范化管理多轮对话流程，系统采用有限状态机（FSM）对对话状态进行建模。状态机包含六个状态，其转换关系如图 4.4 所示。

各状态的定义如下：

WAITING_INPUT（等待输入）：初始状态，系统等待用户输入初始需求。

DETECTING（检测中）：系统已收到需求，正在调用分析智能体进行模糊点检测。

EDITING_AMBIGUITIES（编辑模糊点）：检测完成，用户可查看并编辑模糊点列表。

WAITING_CLARIFICATION（待澄清）：用户已确认模糊点列表，系统正逐个推送问题并等待回答。

CONFIRMED（已确认）：所有模糊点均已澄清，需求规约完备，可进行代码生成。

GENERATING_CODE（生成代码中）：系统正在调用代码生成智能体，状态转换完成后回到 WAITING_INPUT。

协调器（Orchestrator）作为状态机的实现载体，维护核心数据结构包括：当前状态（current_state）、原始需求（original_requirement）、结构化规约（clarified_spec）、问题列表（current_questions）、问题索引（current_question_index）和对话历史（conversation_history）。

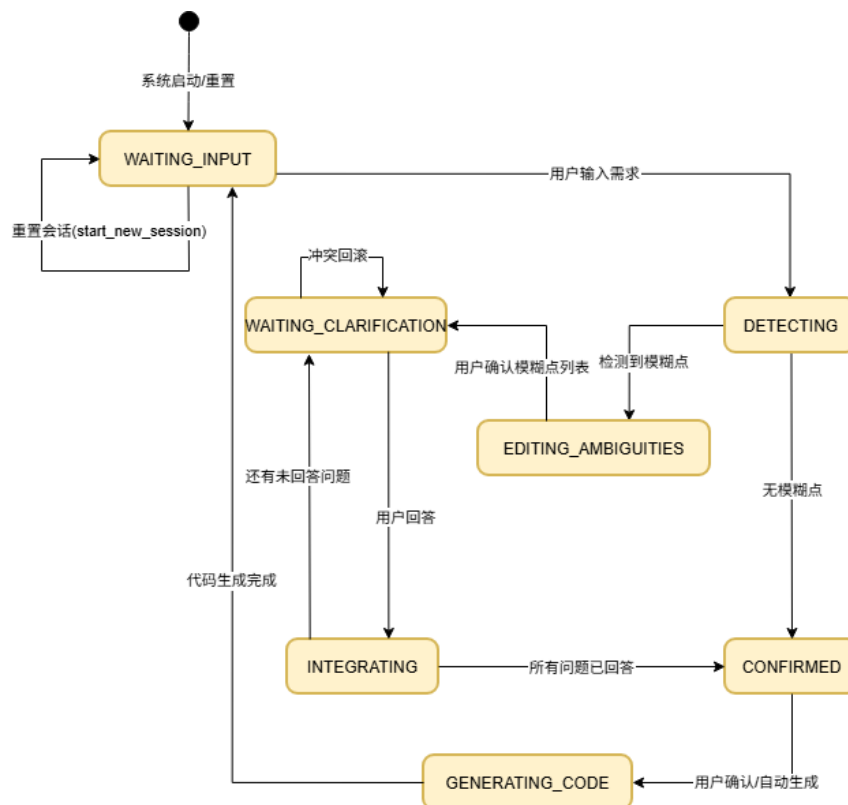


图 4.4 对话管理状态机模型

4.5 数据库设计

4.5.1 概念模型设计

系统的核心实体包括：用户（User）、会话记录（SessionRecord）、代码评分（CodeRating）、高质量案例（HighQualityCase）、需求规格文档（SpecDocument）、模型配置（ModelConfig）、测试结果（TestResult）、澄清模板（ClarificationTemplate）、技能（Skill）、技能-模板关联（SkillTemplate）、测评题目（AssessmentQuestion）、测评记录（AssessmentRecord）。各实体间的 E-R 关系如图 4.5 所示。

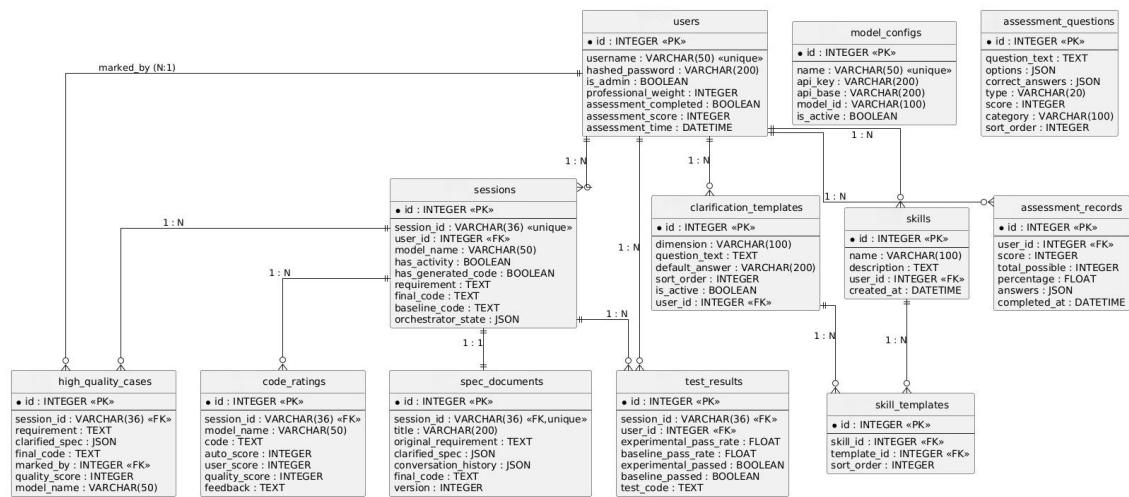


图 4.5 系统数据库 E-R 图

4.5.2 数据库表设计

系统主要数据库表的设计如下列表所示：

1) 用户信息表（users）

用户信息表存储系统用户的基本身份信息，包括用户名、密码哈希和管理员标识。用户既可手动注册，也可由管理员后台创建。表结构如表 4.2 所示。

表 4.2 用户信息表

序号	字段名	数据类型	长度	主键	非空	描述
1	id	INTEGER	-	是	是	用户唯一标识
2	username	VARCHAR	50	否	是	用户登录名
3	hashed_password	VARCHAR	200	否	是	PBKDF2 加密后的密码
4	is_admin	BOOLEAN	-	否	否	是否为管理员
5	professional_weight	INTEGER	-	否	否	专业权重（0-100）
6	assessment_completed	BOOLEAN	-	否	否	是否已完成专业测评
7	assessment_score	INTEGER	-	否	否	测评原始总分

2) 会话记录表（sessions）

会话记录表记录每次需求澄清与代码生成的完整会话信息，包括关联的用户、原始需求、生成代码和协调器状态快照。表结构如表 4.3 所示。

表 4.3 会话记录表

序号	字段名	数据类型	长度	主键	非空	描述
1	id	INTEGER	-	是	是	记录唯一标识
2	session_id	VARCHAR	36	否	是	会话 UUID
3	user_id	INTEGER	-	否	是	关联用户 ID
4	has_activity	BOOLEAN	-	否	否	是否有活跃操作
5	has_generated_code	BOOLEAN	-	否	否	是否已生成代码
6	requirement	TEXT	-	否	否	用户原始需求文本
7	final_code	TEXT	-	否	否	最终生成的代码
8	orchestrator_state	JSON	-	否	否	协调器完整状态快照

3) 模型配置表（model_configs）

模型配置表存储可调用的大语言模型提供商信息，包括 API 密钥和接口地址。管理员可通过界面动态增删和启停模型。表结构如表 4.4 所示。

表 4.4 模型配置表

序号	字段名	数据类型	长度	主键	非空	描述
1	id	INTEGER	-	是	是	配置唯一标识
2	name	VARCHAR	50	否	是	模型名称标识
3	api_key	VARCHAR	200	否	是	API 访问密钥
4	api_base	VARCHAR	200	否	否	API 服务地址
5	is_active	BOOLEAN	-	否	否	是否启用

4) 高质量案例表（high_quality_cases）

高质量案例表存储被用户标记为质量优秀的代码生成案例，用于知识积累和复用。表结构如表 4.5 所示。

表 4.5 高质量案例表

序号	字段名	数据类型	长度	主键	非空	描述
1	id	INTEGER	-	是	是	案例唯一标识
2	session_id	VARCHAR	36	否	是	来源会话 ID
3	requirement	TEXT	-	否	是	原始需求文本
4	clarified_spec	JSON	-	否	是	澄清后的需求规约
5	final_code	TEXT	-	否	是	最终生成的代码
6	marked_by	INTEGER	-	否	否	收录人用户 ID
7	quality_score	INTEGER	-	否	否	质量评分
8	model_name	VARCHAR	50	否	否	使用的模型名称

5) 需求规格文档表 (spec_documents)

需求规格文档表持久化存储每次会话生成的需求规格说明书，包含对话历史、澄清规约和最终代码。表结构如表 4.6 所示。

表 4.6 需求规格文档表

序号	字段名	数据类型	长度	主键	非空	描述
1	id	INTEGER	-	是	是	文档唯一标识
2	session_id	VARCHAR	36	否	是	关联会话 ID
3	title	VARCHAR	200	否	否	文档标题
4	original_requirement	TEXT	-	否	是	原始需求文本
5	clarified_spec	JSON	-	否	是	澄清后的结构化规约
6	conversation_history	JSON	-	否	否	完整对话历史记录
7	final_code	TEXT	-	否	否	最终生成的代码
8	version	INTEGER	-	否	否	文档版本号

6) 代码评分表 (code_ratings)

存储每次代码生成后的系统自动评分和用户手动评分。表结构如表 4.7 所示。

表 4.7 代码评分表

序号	字段名	数据类型	长度	主键	非空	描述
1	id	INTEGER	–	是	是	评分唯一标识
2	session_id	VARCHAR	36	否	是	关联会话 ID
3	model_name	VARCHAR	50	否	是	使用的模型名称
4	code	TEXT	–	否	是	被评分的代码
5	auto_score	INTEGER	–	否	否	系统自动评分（0-100）
6	user_score	INTEGER	–	否	否	用户评分（0-100）
7	quality_score	INTEGER	–	否	否	综合质量分（用于案例库）
8	feedback	TEXT	–	否	否	改进建议

7) 测试结果表（test_results）

存储自动化测试的执行结果，用于实验组与对照组的对比分析。表结构如表 4.8 所示。

表 4.8 测试结果表

序号	字段名	数据类型	长度	主键	非空	描述
1	id	INTEGER	–	是	是	测试结果唯一标识
2	session_id	VARCHAR	36	否	是	关联会话 ID
3	user_id	INTEGER	–	否	是	用户 ID
4	experimental_pass_rate	FLOAT	–	否	是	实验组测试通过率（0-100）
5	baseline_pass_rate	FLOAT	–	否	是	对照组测试通过率
6	experimental_passed	BOOLEAN	–	否	是	实验组是否全部通过（100%为真）
7	baseline_passed	BOOLEAN	–	否	是	对照组是否全部通过
8	test_code	TEXT	–	否	否	生成的测试代码

8) 澄清模板表（clarification_templates）

存储用户自定义的澄清问题模板，支持按维度复用。表结构如表 4.9 所示。

表 4.9 澄清模板表表

序号	字段名	数据类型	长度	主键	非空	描述
1	id	INTEGER	-	是	是	模板唯一标识
2	dimension	VARCHAR	100	否	是	模糊维度名称
3	question_text	TEXT	-	否	是	问题文本（可包含选项）
4	default_answer	VARCHAR	200	否	否	默认答案
5	sort_order	INTEGER	-	否	否	排序序号
6	is_active	BOOLEAN	-	否	否	是否启用
7	user_id	INTEGER	-	否	是	所属用户 ID（数据隔离）

9) 技能表 (skills)

存储用户创建的技能，每个技能是一组澄清模板的集合。表结构如表 4.10 所示。

表 4.10 技能表

序号	字段名	数据类型	长度	主键	非空	描述
1	id	INTEGER	-	是	是	技能唯一标识
2	name	VARCHAR	100	否	是	技能名称
3	description	TEXT	-	否	否	技能描述
4	user_id	INTEGER	-	否	是	所属用户 ID
5	created_at	DATETIME	-	否	否	创建时间

10) 技能-模板关联表 (skill_templates)

实现技能与澄清模板之间的多对多关联。表结构如表 4.11 所示。

表 4.11 技能-模板关联表

序号	字段名	数据类型	长度	主键	非空	描述
1	id	INTEGER	-	是	是	关联唯一标识
2	skill_id	INTEGER	-	否	是	技能 ID
3	template_id	INTEGER	-	否	是	模板 ID
4	sort_order	INTEGER	-	否	否	模板在技能中的顺序

11) 测评题目表 (assessment_questions)

存储专业能力测评的题目，支持单选和多选。表结构如表 4.12 所示。

表 4.12 测评题目表

序号	字段名	数据类型	长度	主键	非空	描述
1	id	INTEGER	–	是	是	题目唯一标识
2	question_text	TEXT	–	否	是	题目文本
3	options	JSON	–	否	是	选项数组
4	correct_answers	JSON	–	否	是	正确答案数组
5	type	VARCHAR	20	否	否	题目类型 (single/multiple)
6	score	INTEGER	–	否	否	该题分值
7	category	VARCHAR	100	否	否	题目分类
8	sort_order	INTEGER	–	否	否	排序序号

12) 测评记录表 (assessment_records)

存储每个用户的测评历史。表结构如表 4.13 所示。

表 4.13 测评记录表

序号	字段名	数据类型	长度	主键	非空	描述
1	id	INTEGER	–	是	是	记录唯一标识
2	user_id	INTEGER	–	否	是	用户 ID
3	score	INTEGER	–	否	是	实际得分
4	total_possible	INTEGER	–	否	是	总分
5	percentage	FLOAT	–	否	是	得分百分比
6	answers	JSON	–	否	是	用户答案

4.6 本章小结

本章从系统架构、功能模块、工作流程、状态机模型和数据库设计五个方面对系统进行了概要设计。首先明确了前后端分离的 B/S 架构模式，划分了表现层、业务逻辑层、AI 服务层和数据持久层四个层次的职责。其次，将系统功能归纳为用户认证、需求澄清、代码生成、历史经验、案例管理和系统管理六大模块。接着，详细描述了从需求输入到代码生成的完整工作流程，以及基于有限状态机的对话管理模型。最后，通过 E-R 图和表结构定义完成了数据库的概念模型设计。本章为后续的系统详细设计与编码实现提供了清晰的架构蓝图和技术规范。

第 5 章 系统详细设计及实现

5.1 多智能体协作框架的实现

本系统的核心创新在于构建了一个基于协调器的多智能体协作框架，通过专业化分工实现需求澄清与代码生成的自动化。该框架由中央协调器（Orchestrator）统一调度五个智能体，各智能体之间的交互关系如图 5.1 所示。

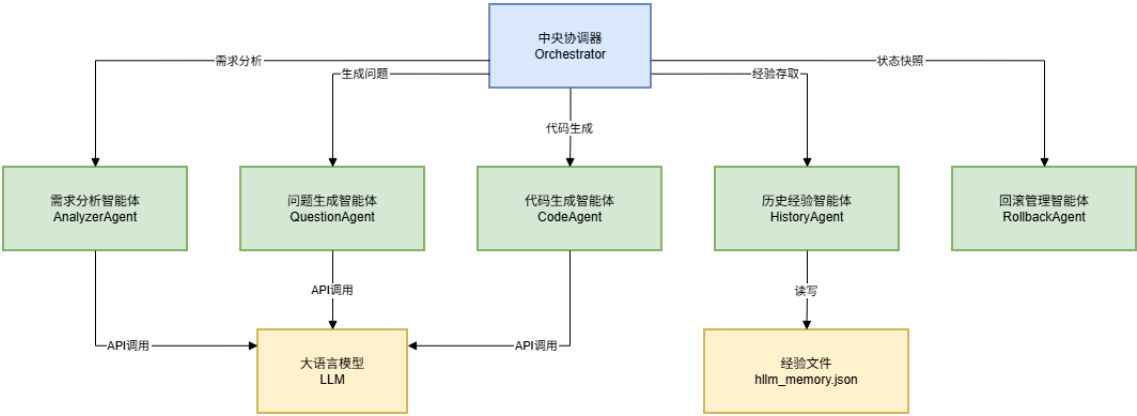


图 5.1 多智能体协作框架示意图

协调器维护对话状态机，在智能体之间传递数据，并管理整个工作流程。各智能体独立封装为单一职责的类，通过协调器的方法调用实现松耦合协作。这种架构将复杂的澄清-生成任务拆解为可独立优化的子任务，既降低了模块间的耦合度，又便于后续对单个智能体进行升级或替换。

以下各节将详细阐述每个核心模块的设计原理与实现细节。

5.2 需求模糊性检测模块（分析智能体）

5.2.1 模块设计原理

需求模糊性检测是本系统的入口模块，其核心功能是分析用户输入的自然语言需求，自动识别其中缺失或模糊的关键信息，并返回结构化的模糊点列表。该

模块的实现借鉴了需求工程中启发式评估方法，采用“开放式反思+结构化核对”的两阶段提示策略。

第一阶段：开放式反思。通过提示词引导 LLM 不设限制地自由思考，从语法、语义、语用等多层面发现潜在模糊之处。提示词设计为：“你是一个严谨的软件需求分析师。根据以下需求，生成一个 JSON 数组，每个元素包含‘dimension’和‘description’字段，描述需求中的模糊点或不明确的细节。”这一阶段利用了 LLM 的零样本推理能力和上下文学习能力，能够捕获意外类型的模糊点。

第二阶段：结构化核对（内置后备机制）。考虑到 LLM 输出存在不确定性，模块内部预定义了通用模糊维度清单作为后备方案。当 LLM 输出无法解析或格式不符合预期时，系统自动降级使用该预定义清单。预定义清单涵盖了需求分析中常见的六个关键维度，如表 5.1 所示。

表 5.1 列出了这六个维度的名称与检查要点，包括：输入规格、输出规格、边界条件、异常处理、性能约束及编程语言/框架。当 LLM 输出的模糊点列表缺失其中任一维度时，系统会自动补充对应的通用检查项，确保模糊点识别的全面性与鲁棒性。

表 5.1 预定义模糊维度检查清单

序号	维度	描述
1	输入规格	输入数据的类型、格式、范围、有效性？
2	输出规格	输出的类型、格式、返回方式？
3	边界条件	输入为空、最大、最小值时的行为？
4	异常处理	如何处理非法输入或运行时错误？
5	性能约束	是否有时间或空间复杂度要求？
6	编程语言/框架	期望使用哪种编程语言或框架？

5.2.2 模块实现

需求模糊性检测模块的核心实现在 AnalyzerAgent 类中。该模块接收用户需求和数据库会话对象，通过调用 LLM 客户端获取分析结果。为保证输出格式的可解析性，提示词中明确要求以 JSON 数组格式输出，并设置了 temperature=0.3 以降

低输出的随机性。响应处理中采用正则表达式提取 JSON 部分，并设置了完整的异常捕获和降级机制。

核心实现代码如表 5.2 所示。

表 5.2 需求模糊性检测核心代码

```
def analyze(self, requirement: str, db: Session) -> List[Dict[str, str]]:
    client = get_llm_client(self.model_name, db)
    # 使用 JSON mode 强制输出结构化数据
    prompt = f"""你是一个严谨的软件需求分析师。根据以下需求，生成一个 JSON 数组，
    每个元素包含"dimension"和"description"字段，描述需求中的模糊点或不明确的细节。
    需求: {requirement}
    输出示例: [{"dimension": "输入规格", "description": "输入数据的类型、格式、范围？"}, ...]
    只输出 JSON 数组，不要有其他文本。"""

    response = client.chat([{"role": "user", "content": prompt}], temperature=0.3)
    # 尝试提取 JSON
    match = re.search(r'([sS]*)', response)
    if match:
        try:
            items = json.loads(match.group())
            if isinstance(items, list) and all("dimension" in i and "description" in i for i in items):
                return items
        except:
            pass
    # 后备: 返回通用检查清单

    return default_ambiguities
```

该实现的关键设计在于：

- 1) JSON 模式强制：提示词中明确要求模型输出 JSON 格式，并提供输出样例供模型参照，提升了解析成功率。
- 2) 双重验证：先用正则表达式提取 JSON 数组部分，再用 json.loads 解析，并对结果进行类型和字段完整性校验。
- 3) 优雅降级：当 LLM 输出不符合预期时，返回预定义的通用检查清单，确保系统始终能为用户提供有价值的模糊点分析。

系统前端模糊点编辑模态框如图 5.2 所示。该模态框以列表形式展示识别出的模糊点，支持用户逐项确认、修改或补充，也可手动新增或删除条目，编辑结果

实时同步至后台。这种可视化交互既保留了 LLM 自动识别的效率，又赋予用户充分的控制权，实现人机协同的需求细化。

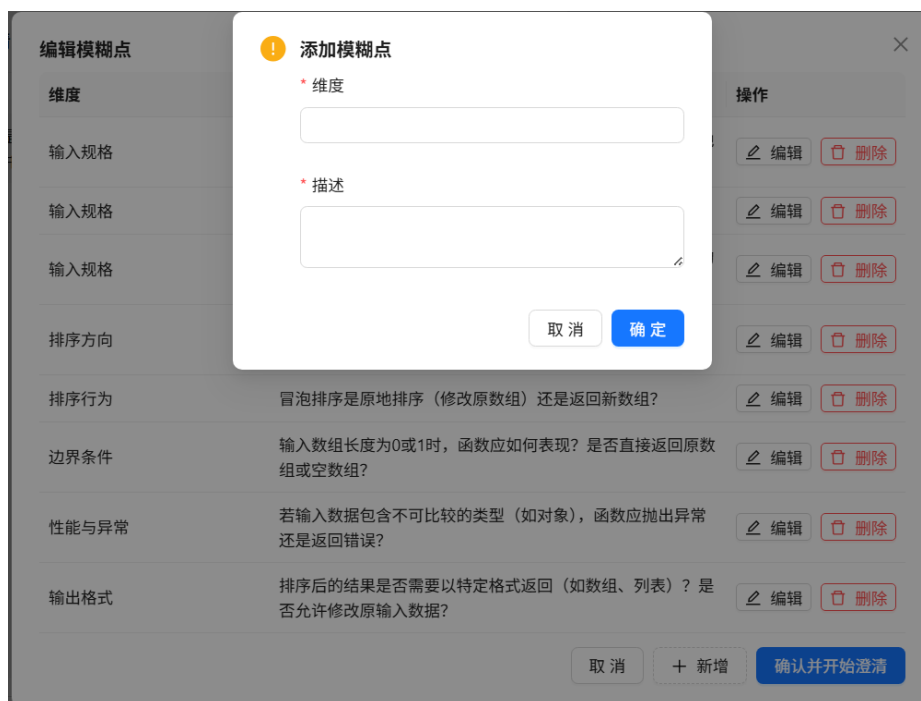


图 5.2 模糊点编辑界面

5.3 澄清问题生成模块（问题生成智能体）

5.3.1 模块设计原理

在获取模糊点列表后，如何将抽象的模糊维度转化为用户易于理解和回答的自然语言问题，是澄清对话能否顺利进行的关键。澄清问题生成模块基于检测出的模糊点列表，为每个模糊维度生成一个具体的引导性问题。模块为每个模糊点独立构造提示词，调用 LLM 生成问题，确保生成的问题与模糊点一一对应。 $\text{temperature}=0.5$ 的设置是在问题表达的多样性和稳定性之间取得平衡——既避免过于随机的输出导致问题偏离主题，又保留一定的灵活性以产生自然流畅的提问。

模块内部流程如图 5.3 所示，该模块独立封装为 QuestionAgent。QuestionAgent 接收模糊点列表后，遍历每个模糊点，动态构造提示词模板调用 DeepSeek LLM，

解析返回文本提取引导性问题，最终汇总返回给用户。该流程实现了“一点一问”的针对性澄清。此外，模块内置异常处理机制：LLM 返回异常时自动使用通用问题模板，确保系统鲁棒性。

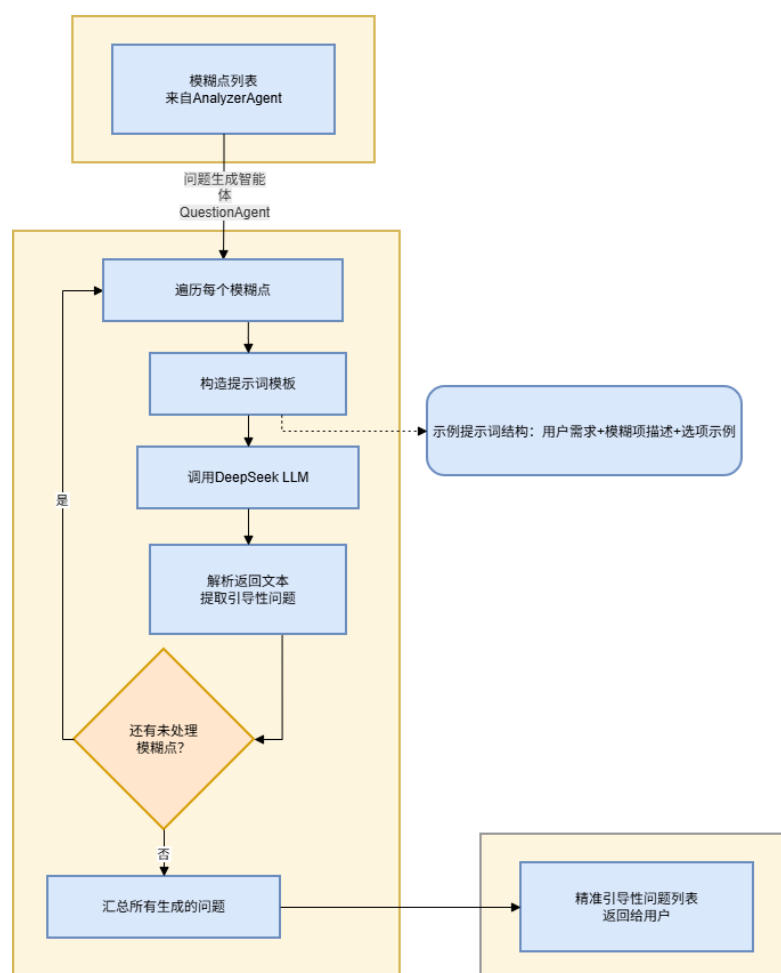


图 5.3 问题生成模块流程图

5.3.2 模块实现

QuestionAgent 类接收用户原始需求、模糊点列表和数据库会话，遍历每个模糊点，提取其维度（dimension）和描述（description），逐个构造提示词调用 LLM 生成问题。生成的问题经 strip() 处理后加入问题列表，最终返回与模糊点一一对应的问题列表。该实现采用逐点独立生成策略，保证了每个模糊点都能获得定制化的引导性提问，避免了多模糊点混合生成时可能出现的遗漏或混淆。该策略的设

计初衷在于，若将所有模糊点一次性输入模型并要求生成对应问题列表，模型可能因上下文过长而忽略某些维度的差异，导致生成的问题同质化或遗漏关键细节；逐点迭代式生成虽然增加了调用次数，但每次聚焦单一维度，能有效提升问题的针对性和区分度。同时，`temperature` 参数设为 0.5，在问题多样性与稳定性之间取得平衡。核心实现代码如表 5.3 所示。

表 5.3 澄清问题生成核心代码

```
def generate(self, requirement: str, ambiguity_list: List[Dict[str, str]], db: Session) -> List[str]:
    client = get_llm_client(self.model_name, db)
    questions = []
    for amb in ambiguity_list:
        dim = amb.get("dimension", "未知")
        desc = amb.get("description", "")
        prompt = f"""需求：{requirement}
关于{dim}不明确：{desc}
请生成一个具体的引导性问题，提供选项或示例。每个选项后换行。只输出问题。"""
        q = client.chat([{"role": "user", "content": prompt}], temperature=0.5)
        questions.append(q.strip())
    return questions
```

提示词设计中要求“提供选项或示例”，使生成的问题具有引导性，降低用户的认知负担——用户可以直接从选项中选择，而非从零开始构思答案。例如，对于“输入规格”模糊点，模型可能生成：“输入数据的类型是什么？ A. 整数列表； B. 字符串列表； C. 浮点数列表； D. 混合类型列表”，用户只需回答字母即可。`temperature=0.5` 的设置经过实验对比得出：过低（如 0.1）会导致问题措辞机械，缺乏引导性选项的多样性和自然度；过高（如 0.8）则可能引入偏离主题的随机变体，削弱“提供选项”这一指令的执行效果。0.5 的取值在保持问题结构清晰、紧扣需求的同时，允许适度的语言变化，使用户体验更为自然流畅。

系统前端澄清问题界面如图 5.4 所示。界面左侧以对话气泡形式逐条展示引导性问题，用户点击选项按钮即可回答，也支持文字回答。

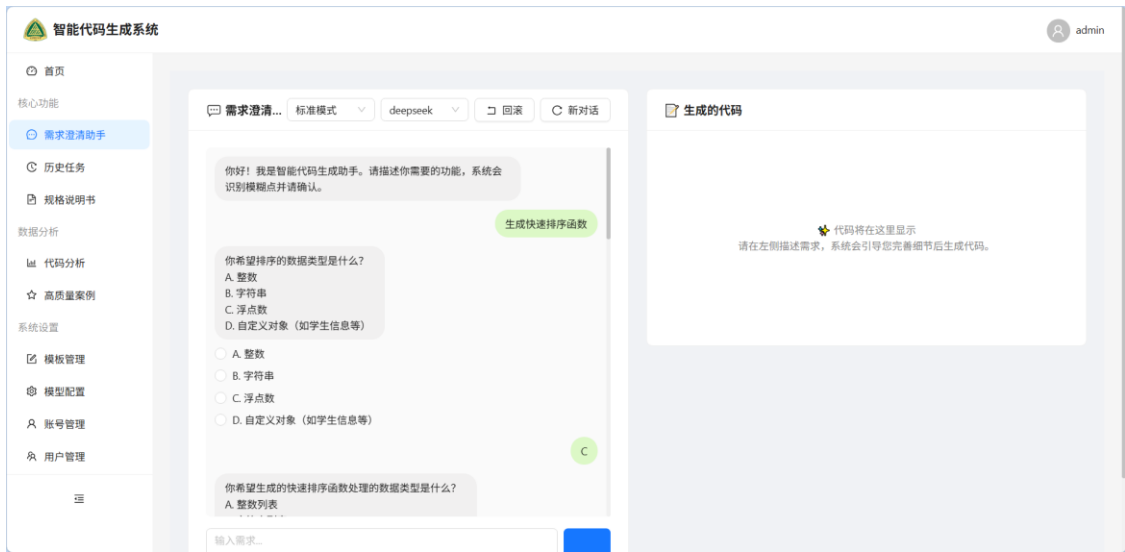


图 5.4 澄清问题界面

5.4 多轮对话管理与回滚机制（协调器+回滚智能体）

5.4.1 对话状态管理

对话状态管理是多轮交互有序推进的基础，它确保系统在任何时刻都能明确当前进度和下一步操作，避免对话流程混乱。在多轮澄清场景中，用户与系统之间的交互是一个逐步收敛的过程——每轮问答都可能影响后续提问的方向和内容，因此需要一种可靠的机制来记录对话的演进轨迹，使得系统行为可预测、可回溯。中央协调器（Orchestrator）是整个系统的大脑，负责编排所有智能体的协作流程。其内部维护一个有限状态机，通过 `current_state` 字段追踪当前对话所处的阶段，通过 `clarified_spec` 字典积累用户逐步确认的需求细节，通过 `current_question_index` 指针控制问题的顺序推送。这些要素协同工作，使协调器能够在“接收需求—检测模糊点—生成问题—收集答案—确认规约—生成代码”的完整生命周期中，精确控制状态的转换条件与时机，确保每一轮交互都有清晰的上下文依据。

协调器的核心状态结构及方法如表 5.4 所示。

系统对话历史截图（部分）如图 5.5 所示。

表 5.4 协调器核心数据结构

字段/方法	类型	描述
state	ConversationState	当前对话状态（枚举值）
original_requirement	str	用户输入的原始需求文本
clarified_spec	Dict[str, str]	澄清后的结构化需求规约，键为维度名，值为用户答案
current_questions	List[str]	当前待回答的问题列表
current_question_index	int	当前正在提问的索引
ambiguity_list	List[Dict]	检测到的模糊点列表
conversation_history	List[Dict]	完整对话历史记录
receive_requirement()	方法	接收需求，触发模糊点检测和历史经验检索
receive_answer()	方法	接收回答，执行冲突检测和状态推进
generate_code()	方法	调度代码生成和历史经验记录

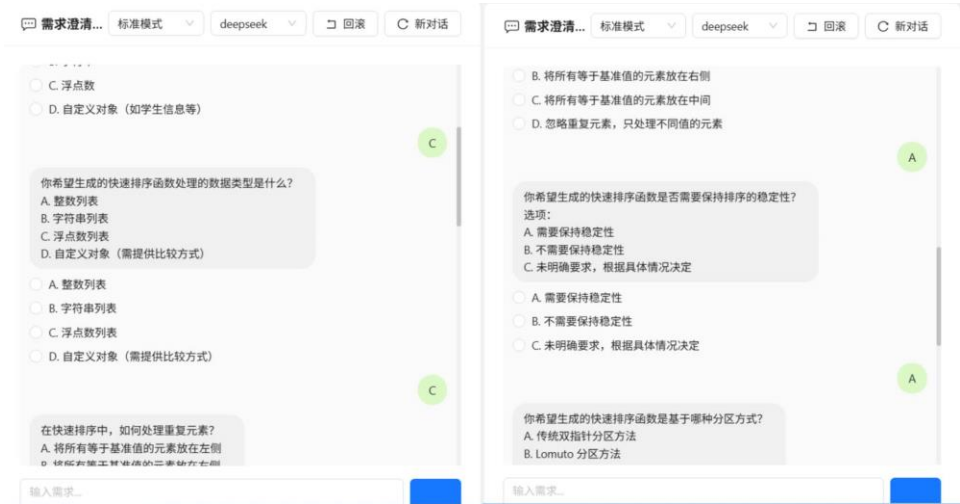


图 5.5 部分对话历史截图

5.4.2 答案规范化处理

由于用户回答方式多样（字母选择、短语、完整句子），系统需要将用户答案统一转换为可存储到规约中的规范文本。若直接将原始回答存入规约，后续代码生成模块将面临语义不一致的输入，可能影响生成代码的准确性。为此，协调器实现了_normalize_answer 方法：若用户输入为单个字符（如“A”、“b”），则尝试从当前问题文本中匹配对应选项的完整描述；若匹配失败或输入为多字符，则直接使用用户的原始输入。

规范化处理的核心逻辑如表 5.5 所示。

表 5.5 答案规范化核心代码

<pre>def _normalize_answer(self, answer: str, question_text: str) -> str: """将答案转换为完整文本，若答案为单个字母则尝试匹配选项描述""" ans = answer.strip() if not re.match(r'^[a-zA-Z0-9]\$', ans): return ans.lower() if question_text: pattern = r'(?:\n)\s*' + re.escape(ans) + r'[\s]+([\n]+)' match = re.search(pattern, question_text, re.MULTILINE) if match: option_text = match.group(1).strip() option_text = re.split(r'[,.]', option_text)[0].strip() if option_text: return option_text return ans.lower()</pre>

5.4.3 回滚机制

回滚机制是本系统保证需求规约一致性的关键设计。当用户在澄清过程中希望修改之前已确认的答案时，若简单地覆盖旧值，可能导致后续问题失去意义。因此，系统采用状态快照回滚策略，支持用户主动回退到任意历史状态并重新澄清。

回滚管理的完整流程如图 5.6 所示。

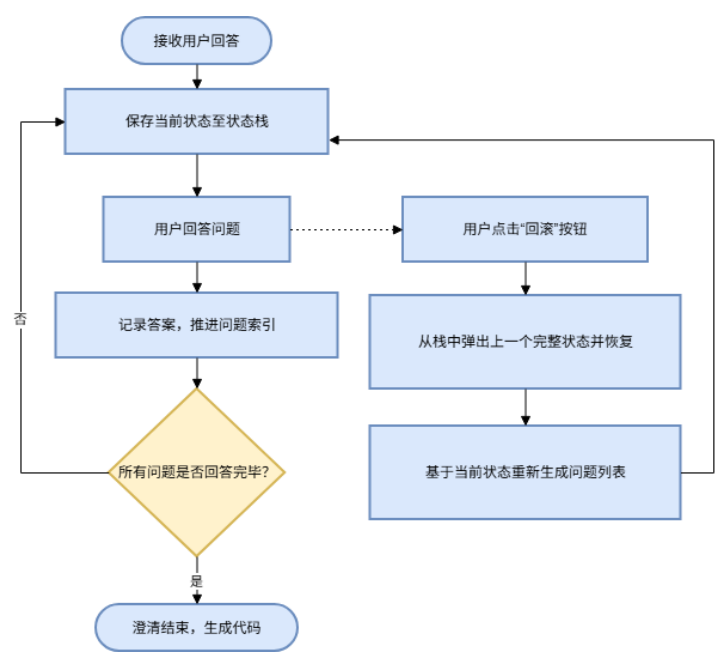


图 5.6 回滚流程图

回滚管理智能体（RollbackAgent）维护一个最大容量为 10 的状态栈。每次用户回答前，协调器将当前完整状态（包括对话状态、原始需求、规约快照、问题索引）压入栈中。用户回答后，系统正常推进问题索引。当用户需要修改先前答案时，可点击前端“回滚”按钮，协调器从栈中弹出上一个完整状态并恢复，然后基于原始需求和新答案重新生成问题列表。

回滚的核心实现如表 5.6 所示。表 5.6 中的 `receive_answer` 方法负责保存快照、记录答案并推进问题；回滚时调用 `rollback.pop()` 获取历史状态并执行 `restore_from_state()` 进行完整恢复。该设计保证了用户可以在任意时刻通过回滚操作修正之前的答案，且不会导致规约数据出现不一致。

系统前端回滚操作界面如图 5.7 所示，用户可在对话中随时点击“回滚”按钮返回上一步重新作答，提升了交互的容错性。

表 5.6 回滚核心代码

```
def receive_answer(self, answer: str, db: Session) -> Tuple[Optional[str], bool]:
    # 保存快照用于可能的回滚
    self.rollback.push(self.get_full_state(), self.original_requirement,
                       self.clarified_spec.copy(), self.current_question_index)

    # 规范化答案并记录
    current_q = self.current_questions[self.current_question_index]
    norm_ans = self._normalize_answer(answer, current_q)
    key = self.ambiguity_list[self.current_question_index].get("dimension")
    self.clarified_spec[key] = norm_ans

    # 推进问题索引
    self.current_question_index += 1

    # 判断是否还有未回答的问题
    if self.current_question_index < len(self.current_questions):
        return self.current_questions[self.current_question_index], False
    else:
        self.state = ConversationState.CONFIRMED
        return None, True
```

关于输入数据的类型和范围，您希望基数排序函数支持以下哪种情况？

1. 仅支持非负整数数组，例如 [170, 45, 75, 90]

2. 支持包含负数的整数数组，例如 [-5, 3, -12, 0, 8]

3. 支持浮点数数组，例如 [3.14, 2.71, 1.41]

4. 支持字符串数组，按字典序排序，例如 ["cat", "dog", "apple"]

☐ 1. 仅支持非负整数数组，例如 [170, 45, 75, 90]

☐ 2. 支持包含负数的整数数组，例如 [-5, 3, -12, 0, 8]

☐ 3. 支持浮点数数组，例如 [3.14, 2.71, 1.41]

☐ 4. 支持字符串数组，按字典序排序，例如 ["cat", "dog", "apple"]

仅支持非负整数数组

关于排序顺序，您希望基数排序按以下哪种方式执行？

A. 升序（从小到大，例如 [1, 2, 3, 4, 5]

B. 降序（从大到小，例如 [5, 4, 3, 2, 1]

C. 由用户传入一个参数来控制升序或降序

☐ A. 升序（从小到大，例如 [1, 2, 3, 4, 5]

☐ B. 降序（从大到小，例如 [5, 4, 3, 2, 1]

☐ C. 由用户传入一个参数来控制升序或降序

☒ 请回答：关于输入数据的类型和范围，您希望基数排序函数支持以下哪种情况？

1. 仅支持非负整数数组，例如 [170, 45, 75, 90]

2. 支持包含负数的整数数组，例如 [-5, 3, -12, 0, 8]

3. 支持浮点数数组，例如 [3.14, 2.71, 1.41]

4. 支持字符串数组，按字典序排序，例如 ["cat", "dog", "apple"]

☐ 1. 仅支持非负整数数组，例如 [170, 45, 75, 90]

☐ 2. 支持包含负数的整数数组，例如 [-5, 3, -12, 0, 8]

☐ 3. 支持浮点数数组，例如 [3.14, 2.71, 1.41]

☐ 4. 支持字符串数组，按字典序排序，例如 ["cat", "dog", "apple"]

请输入你的回答...

发送

图 5.7 回滚操作界面

5.4.4 状态持久化与会话恢复

为支持断点续传和长期存储，协调器实现了 `get_full_state()` 和 `restore_from_state()` 两个方法，可将当前内存状态序列化为 JSON 兼容的字典，存入数据库的 `orchestrator_state` 字段。状态快照包含对话状态枚举值（以整数存储）、原始需求、澄清规约、问题列表、问题索引、模糊点列表、对话历史、生成模式、上次生成的代码及模型名称等完整信息。这两个方法互为逆操作，确保状态能够完整保存并准确还原。

状态持久化与会话恢复的核心实现如表 5.7 所示。其中 `get_full_state()` 返回一个包含所有关键字段的字典，而 `restore_from_state()` 则根据传入的状态字典逐一恢复协调器的各属性，包括将整数形式的对话状态转换回对应的枚举类型。

55

当用户刷新页面或重新登录后，前端通过会话 ID 请求会话状态接口，后端从数据库读取持久化的状态快照并调用 `restore_from_state()` 重建协调器实例，实现无缝的会话恢复体验。

表 5.7 状态持久化与会话恢复核心代码

```
def get_full_state(self) -> Dict:
    return {
        "state": self.state.value, # 枚举转整数
        "original_requirement": self.original_requirement,
        "clarified_spec": self.clarified_spec.copy(),
        "current_questions": self.current_questions.copy(),
        "current_question_index": self.current_question_index,
        "ambiguity_list": self.ambiguity_list.copy(),
        "conversation_history": self.conversation_history.copy(),
        "generation_mode": self.generation_mode,
        "last_generated_code": self.last_generated_code,
        "model_name": self.model_name
    }
def restore_from_state(self, state: Dict):
    self.state = ConversationState(state["state"]) # 整数转回枚举
    self.original_requirement = state["original_requirement"]
    self.clarified_spec = state["clarified_spec"].copy()
    # ... 其余字段逐一恢复
```

5.5 历史经验学习机制（历史经验智能体）

5.5.1 机制设计原理

历史经验学习机制（HLLM）是本系统实现“从经验中学习”的核心设计。其基本原理是将每次成功完成的需求澄清-代码生成过程作为一个经验案例进行持久化存储，并在后续相似需求出现时检索复用。

HistoryAgent 负责管理这些历史经验的存储和检索。经验数据以 JSON 格式持久化存储在 `hllm_memory.json` 文件中，每条记录包含原始需求（`requirement`）、澄清后的结构化规约（`clarified_spec`）、最终生成的代码（`final_code`）和质量评分（`quality_score`）四个字段。

5.5.2 模块实现

HistoryAgent 在初始化时自动加载已存储的经验文件，提供 record_success 和 get_similar 两个核心方法。

1) 经验记录。当代码生成成功后，协调器调用 record_success 方法，将本次交互的需求、规约和代码作为一条经验记录追加到内存列表中，并持久化保存到 JSON 文件。经验记录核心实现如表 5.8 所示。

2) 相似检索。用户提交新需求时，协调器调用 get_similar 方法，基于关键词重叠度从历史经验库中检索最相似的案例。检索算法将需求文本分词后计算词集合的 Jaccard 相似度，返回相似度最高且大于零的案例，核心实现如表 5.9 所示。

表 5.8 经验记录核心代码

```
def record_success(self, requirement: str, clarified_spec: Dict, final_code: str, quality_score: float = 1.0):
    self.memory.append({
        "requirement": requirement,
        "clarified_spec": clarified_spec,
        "final_code": final_code,
        "quality_score": quality_score
    })
    self._save()
```

表 5.9 相似经验核心代码

```
def get_similar(self, requirement: str) -> Optional[Dict]:
    words = set(requirement.lower().split())
    if not words:
        return None
    best = None
    best_score = 0
    for rec in self.memory:
        score = len(words & set(rec["requirement"].lower().split()))
        if score > best_score:
            best_score = score
            best = rec
    return best if best_score > 0 else None
```

5.6 代码生成与安全检测模块（代码生成智能体）

5.6.1 模块设计原理

代码生成模块基于澄清后的完整需求规约，生成高质量的目标代码及其解释。通过 `temperature=0.2` 的低温度设置确保代码输出的确定性和一致性。

5.6.2 模块实现

1) 代码实现

CodeAgent 类的 `generate` 方法接收原始需求、澄清规约字典、数据库会话、生成模式和可选相似代码作为参数。方法内部将澄清规约格式化为提示词详情部分，根据模式选择不同的指令文本，若存在相似案例代码则将前 1500 字符作为参考注入提示词。生成的响应通过正则表达式提取代码块，并去除可能的 Markdown 标记。

代码生成后，系统自动弹出评分对话框，显示 LLM 自动评分结果，并邀请用户输入主观评分（0-100 分）。若用户评分达到 85 分及以上，系统提示是否收录到高质量案例库。这一评分反馈机制有助于积累高质量训练数据，为后续模型优化提供依据。系统前端代码生成与评分界面如图 5.8 所示。

核心实现代码如表 5.10 所示。



图 5.8 代码生成与评分界面

表 5.10 代码生成核心代码

```

def generate(self, original_requirement: str, clarified_spec: Dict[str, str], db: Session,
             mode: str = "standard", similar_code: Optional[str] = None) -> Tuple[str, str]:
    client = get_llm_client(self.model_name, db)
    details = "\n".join([f"- {k}: {v}" for k, v in clarified_spec.items()])
    mode_instruction = {
        "standard": "生成健壮的代码，包含错误处理和输入校验。",
        "explanatory": "为每一行关键代码添加详细的中文注释。",
        "minimal": "仅实现核心功能，不包含任何错误处理或输入校验。"
    }.get(mode, "生成健壮的代码。")

    similar_section = ""
    if similar_code:
        similar_section = f"\n 参考相似案例代码：\n```\n{similar_code[:1500]}\n```\n 请在此基础上根据当前需求调整。"

    prompt = f"""原始需求：{original_requirement}
澄清细节：
{details}
要求：{mode_instruction}
{similar_section}
只输出代码，用```语言名 ... ```包裹。"""
    response = client.chat([{"role": "user", "content": prompt}], temperature=0.2,
                           max_tokens=4096)
    match = re.search(r'```(?:\w+)?\s*(.*?)\n```', response, re.DOTALL)
    code = match.group(1).strip() if match else response.strip()
    code = re.sub(r'\s*\*\s*\*\s*\*\s*', "", code)
    return code, "代码已生成"

```

2) 自动化测试与对比

系统设计了自动化测试模块，用于对实验组与对照组生成的代码进行统一质量评估。该模块集成在 CodeAgent 中，通过 run_automated_test 方法完成从可测试性检查到结果存储的全流程。核心步骤包括：

1. 可测试性检查：利用 Python 的 ast 模块解析代码，要求代码至少定义一个函数，且不得包含 pygame、tkinter、flask 等 GUI/Web 框架关键字。
2. 测试代码生成：调用 LLM 生成 pytest 测试代码，强制首行为 from solution import*，并明确禁止生成任何包含文件操作、网络请求、系统调用或危险函数（如 eval、exec）的测试代码。

3.测试执行：在临时目录中写入 solution.py 和 test_solution.py，使用 subprocess 运行 pytest，统计通过率：100%为“通过”，否则为“未通过”。

4.结果保存：调用/api/test_results/save 接口存储通过率及通过标志，供后续在 Analysis.js 页面中以柱状图形式展示实验组与对照组的总体通过率对比。

前端一键测试功能由 CodeGen.js 中的 runBothTests 函数实现：该函数先对实验组和对照组的代码分别执行可测试性检查，然后并行生成测试代码并执行，最后将每个代码块的通过率及详细测试结果展示在代码区域下方。

核心实现代码如表 5.11 所示。

系统前端测试分析界面如图 5.9 所示。

表 5.11 自动化测试核心代码

```
def generate_test_code(req: GenerateTestRequest, current_user: User = Depends(get_current_user),
db: Session = Depends(get_db)):
    func_names = extract_function_names(req.code)
    if not func_names:
        raise HTTPException(400, "代码中未定义任何函数，无法生成测试")
    function_hint = f"待测代码中定义了以下函数：{' '.join(func_names)}。请针对这些函数编写测试，重点测试主要功能函数。"
    security_warning = """注意：生成的测试代码只能使用 pytest 和标准库，禁止包含任何文件操作（如 open、write）、网络请求（如 requests）、系统调用（如 os.system、subprocess）或危险函数（如 eval、exec）。"""
    try:
        client = get_llm_client("deepseek", db)
        prompt = f"""你是一个严谨的测试工程师。根据以下需求和代码，生成完整的 pytest 测试代码。
{function_hint}
{security_warning}
要求：
1. 测试代码首行必须为 `from solution import *`。
2. 分析代码中每个函数的错误处理方式：
    - 如果函数使用 `raise` 抛出异常（例如 ValueError, TypeError），则测试非法输入时必须使用 `pytest.raises` 断言异常。
    - 如果函数返回特殊值（如 None、-1）表示错误，则测试应断言返回值等于该特殊值。
3. 测试必须覆盖正常情况、边界情况（空列表、零、None 等）以及异常情况。
4. 只输出纯 Python 测试代码，不要任何解释或 markdown 标记。
5. 确保测试可独立运行，仅依赖 pytest 和 solution 模块。
需求：{req.requirement}
待测代码：
{req.code}"""
```



图 5.9 测试分析界面

5.6.3 安全检测

代码生成完成后，协调器对生成的代码执行基于关键词的安全检测。检测规则包括：

- 1) 硬编码凭据检测：若代码包含“password”或“secret”关键词，提示“检测到敏感关键词，请勿硬编码凭据”。
- 2) 代码注入风险检测：若代码包含“exec(”或“eval(”调用，提示“使用了exec/eval，存在代码注入风险”。
- 3) SQL 注入风险检测：若代码同时包含“sql”和“+”运算符且后跟“sql”，提示“检测到 SQL 字符串拼接，建议使用参数化查”。核心代码如表 5.12 所示。

这些安全建议会随代码一同返回前端，以警告提示的形式展示，帮助用户识别和防范潜在的安全风险。系统前端安全警告提示如图 5.10 所示。

表 5.12 安全检测核心代码

```
suggestions = []
if "password" in code.lower() or "secret" in code.lower():
    suggestions.append("检测到敏感关键词，请勿硬编码凭据。")
if "exec(" in code or "eval(" in code:
    suggestions.append("使用了 exec/eval，存在代码注入风险。")
if "sql" in code.lower() and "+" in code and "sql" in code.lower():
    suggestions.append("检测到 SQL 字符串拼接，建议使用参数化查询。")
```



图 5.10 安全警告提示

5.7 前端交互实现与完整功能界面

5.7.1 核心交互逻辑

前端基于 React 框架构建，会话状态通过 `sessionStorage` 进行持久化缓存，确保用户刷新页面后能够恢复之前的对话上下文而不丢失进度。核心交互逻辑围绕 `awaitingAnswer` 状态变量展开，用于区分用户当前是在输入新需求还是回答澄清问题。这种状态驱动的交互模式将多轮对话的复杂性收敛为简单的二值判断，使前端能够以统一的接口适配后端不同阶段的响应，无需引入额外的路由或页面跳转。发送消息时，前端根据该状态决定调用 `/api/detect` 接口（需求检测）还是 `/api/answer` 接口（提交答案）。当后端返回 `ready` 标志为 `true` 时，前端将生成的代码和解释渲染到代码展示区域；否则，将新问题追加到消息列表并推送。在异常处理路径中，前端会捕获网络错误或后端返回的业务异常，通过 `message.error` 向用户展示友好提示，并自动将 `awaitingAnswer` 重置为 `true` 以恢复交互能力，避免用户因单次请求失败而陷入无法操作的状态。

核心交互逻辑代码如表 5.13 所示。

表 5.13 前端核心交互逻辑代码

```
const sendAnswer=async()=>{
  if(!inputValue.trim()||loading)return;
  if(!awaitingAnswer)return sendRequirement();
  setMessages(p=>[...p,{role:'user',content:inputValue}]);
  const a=inputValue.trim();
  setInputValue('');setLoading(true);setAwaitingAnswer(false);
  try{
    const r=await axios.post('/api/answer',{answer:a,session_id:sessionId});
    if(r.data.ready){
      setCode(r.data.code);setExplanation(r.data.explanation);
      setSuggestions(r.data.security_suggestions||[]);
      setIsCodeGenerated(true);
      setMessages(p=>[...p,{role:'assistant',content:'代码已生成！'}]);
    }else{
      setMessages(p=>[...p,{role:'assistant',content:r.data.question}]);
      setAwaitingAnswer(true);
    }
  }catch(e){
    message.error('处理失败: '+ (e.response?.data?.detail||e.message));
    setAwaitingAnswer(true);
  }finally{setLoading(false);}
};
```

5.7.2 用户认证与系统入口

系统提供独立的登录和注册页面，用户输入用户名和密码后登录。注册页面结构类似，额外包含确认密码字段。用户的注册登录页面如图 5.11、5.12 所示。



图 5.11 注册页面

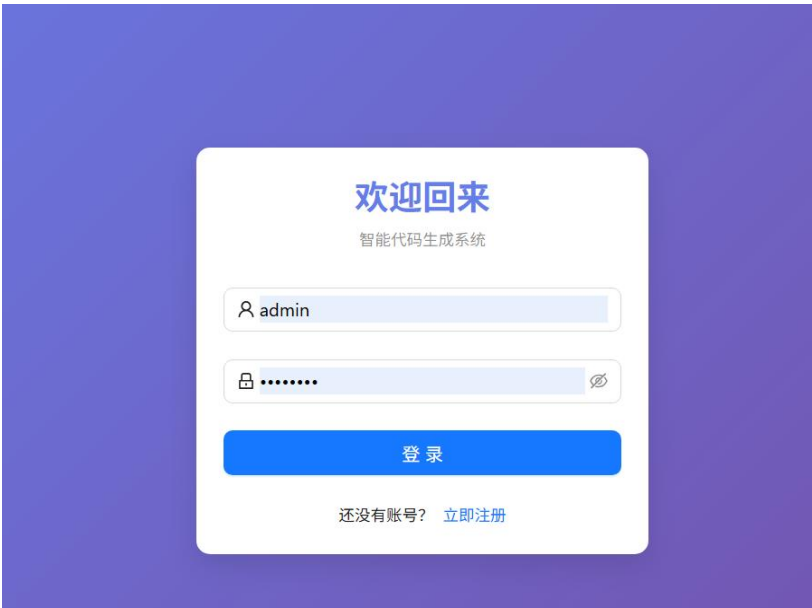


图 5.12 登录页面

登录后进入的主界面如图 5.13 所示。系统前端采用单页应用架构，所有功能模块通过侧边栏导航访问，侧边栏分为“核心功能”“数据分析”“系统设置”三大分组；中间内容区默认展示仪表盘；顶部栏右侧显示用户头像和用户名下拉菜单，下拉菜单包含“个人中心”和“退出登录”。

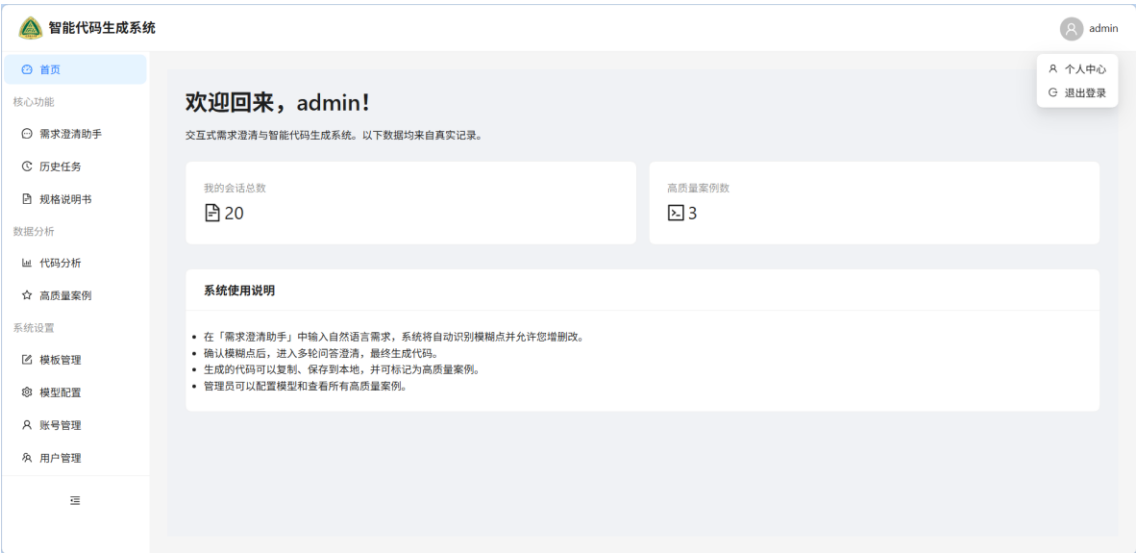


图 5.13 系统主界面

5.7.3 历史任务与规格说明书管理

点击侧边栏“历史任务”进入 History 页面，如图 5.14 所示。表格展示所有已生成代码的会话，列包括原始需求、系统评分、用户评分及操作按钮。操作按钮包含“查询”（按需求内容搜索）、“详情”、“收录案例”和“删除”。点击“详情”弹出模态框，显示完整的对话历史、原始需求和最终代码，并支持保存代码文件。列表中每条记录还支持一键收录高质量案例（“收录案例”按钮），便于快速扩充案例库。表格上方提供搜索框，支持按原始需求关键词快速筛选任务。分页组件便于浏览大量历史记录，每页默认显示 10 条。点击“收录案例”可直接将当前记录标记为高质量案例并存入案例库。

点击“规格说明书”进入 Specifications 页面，如图 5.15 所示。表格展示已保存的规格文档（会话 ID、标题），支持“查询”“查看”“编辑”“删除”。编辑功能允许用户直接修改标题和澄清规格（JSON 格式），保存后更新数据库。编辑界面如图 5.15 下半部分所示，包含标题输入框和 JSON 格式的澄清规格编辑器，用户可直接修改规格内容，并通过“取消”或“保存”按钮完成操作。保存后的数据将同步更新至数据库中的对应规格文档。JSON 编辑器提供语法高亮和实时校验，格式错误时给予提示。编辑保存前会校验 JSON 合法性，防止无效数据入库。查看模式以只读形式展示规格详情，避免误操作。

智能代码生成系统

admin

首页

需求澄清助手

历史任务

规格说明书

数据分析

代码分析

高质量案例

系统设置

模板管理

模型配置

账号管理

用户管理

历史任务追溯

搜索需求内容

原始需求	系统评分	用户评分	操作
生成冒泡排序函数	85	89	详情 收录案例 删除
生成一个图书管理系统	75	75	详情 收录案例 删除
生成快速排序函数	85	88	详情 收录案例 删除
生成两个整数相加的函数	95	94	详情 收录案例 删除
生成消消乐小游戏	65	65	详情 收录案例 删除
生成俄罗斯小游戏	75	75	详情 收录案例 删除
生成贪吃蛇小游戏	85	85	详情 收录案例 删除
生成一个飞机小游戏	75	75	详情 收录案例 删除

图 5.14 历史任务列表



图 5.15 规格说明书列表及编辑界面

5.7.4 代码分析可视化

分析页面（Analysis）为用户提供代码质量的统计图表，如图 5.16 所示。页面包含三个部分：

- 1）顶部统计卡片：已评估记录总数、整体平均分。
- 2）中部折线图：最近 10 次评分的趋势，横轴为任务序号，纵轴为评分，可直观反映评分波动情况。
- 3）底部柱状图：各模型（deepseek、doubao、qwen 等）的平均分对比，便于横向比较性能。

所有图表数据均基于历史任务中已评分的会话自动计算，页面支持手动刷新以获取最新统计结果。

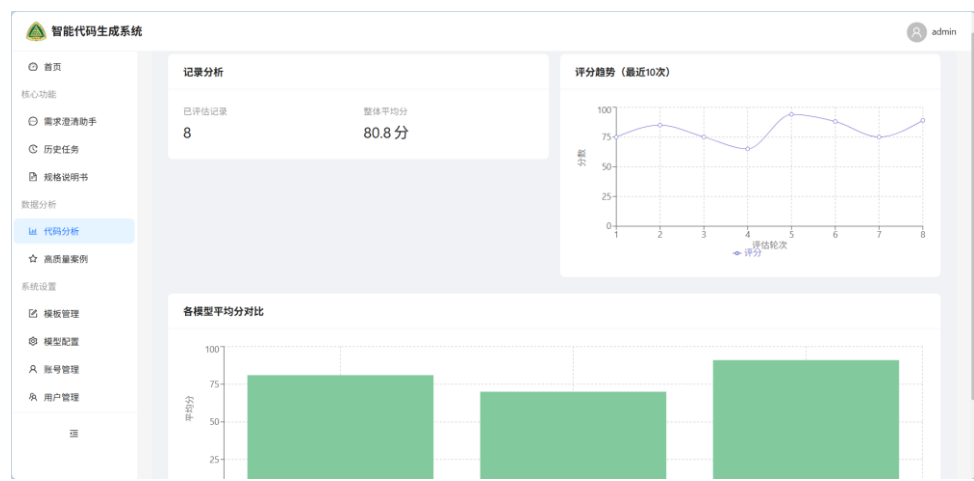


图 5.16 代码分析页面

5.7.5 高质量案例库

高质量案例库是历史经验学习机制的用户可见界面，如图 5.17 所示。用户可以在此查看所有被收录的高质量案例（管理员可以看到所有用户的高质量案例）。表格展示案例 ID、需求摘要、模型名称、质量分，每行提供“预览”“保存”按钮。点击“预览”按钮弹出模态框，显示完整的代码内容，并支持保存到本地。

此外，界面顶部提供搜索框（支持按需求内容关键词检索），表格支持按 ID、质量分等字段排序，方便用户快速定位所需案例。每行右侧的“保存”按钮允许用户将当前案例收录至个人收藏库（管理员可将任何用户的高质量案例标记为全局推荐案例），“删除”按钮仅对管理员或案例提交者可见，用于移除不符合质量标准的案例，确保案例库的权威性与实用性。

当用户从历史任务列表中选择一条记录并点击“保存至案例库”时，系统会校验是否已存在相似案例，避免重复收录；若质量分超过预设阈值（如 90 分），系统还会自动弹窗提醒用户将其加入高质量案例库，实现被动记录与主动收集的结合。

该界面与 5.5 节描述的历史经验学习机制相辅相成：后端自动记录高分案例，前端允许用户手动收录或删除，共同构建系统的经验积累与复用闭环。

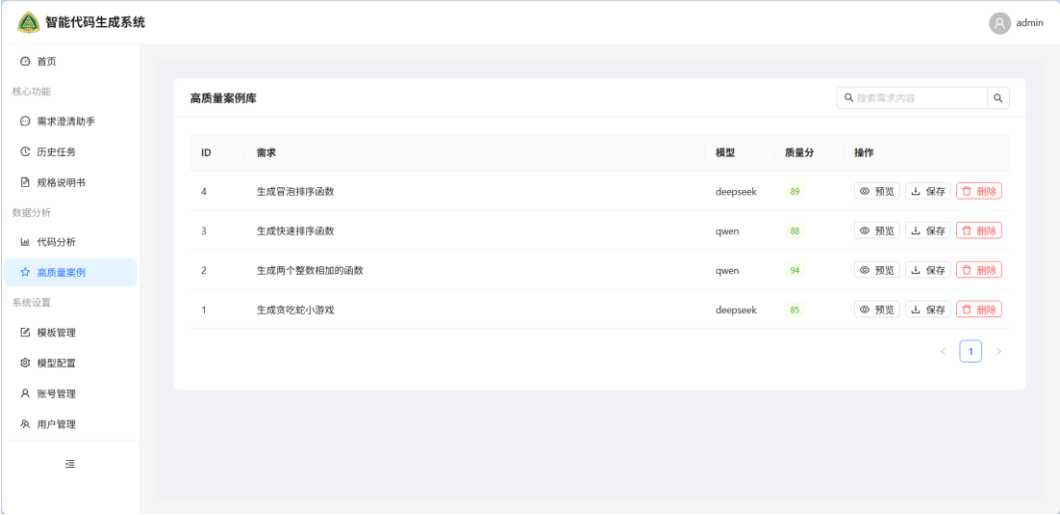


图 5.17 高质量案例库界面

5.7.6 技能管理

1）模板管理支持用户自定义澄清问题模板（按 user_id 隔离）。界面如图 5.18 所示，表格列包括维度、问题文本、默认答案、排序、状态；用户可新增、编辑、删除模板，新增/编辑模态框包含上述字段及启用开关。

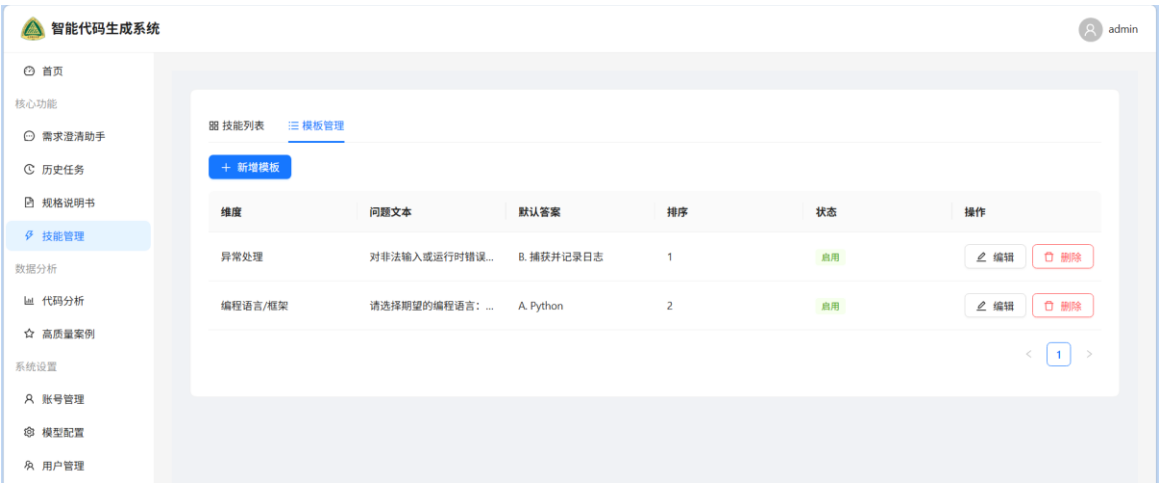


图 5.18 模板管理界面

2) 技能管理用于集成自定义澄清问题模板为特定场景，可直接进行应用。界面如图 5.19 所示，用户可新建、编辑、删除技能以及关联模板。



图 5.19 技能列表界面

5.7.7 账号管理

用户点击顶部下拉菜单中的“个人中心”进入 Profile 页面，如图 5.20 所示。页面包含两个卡片：“账号信息”（显示用户名和角色）和“修改密码”（包含原密码、新密码、确认新密码表单）。修改密码成功后系统会强制重新登录。表单支持前端实时验证，新密码与确认密码不一致时给予提示。

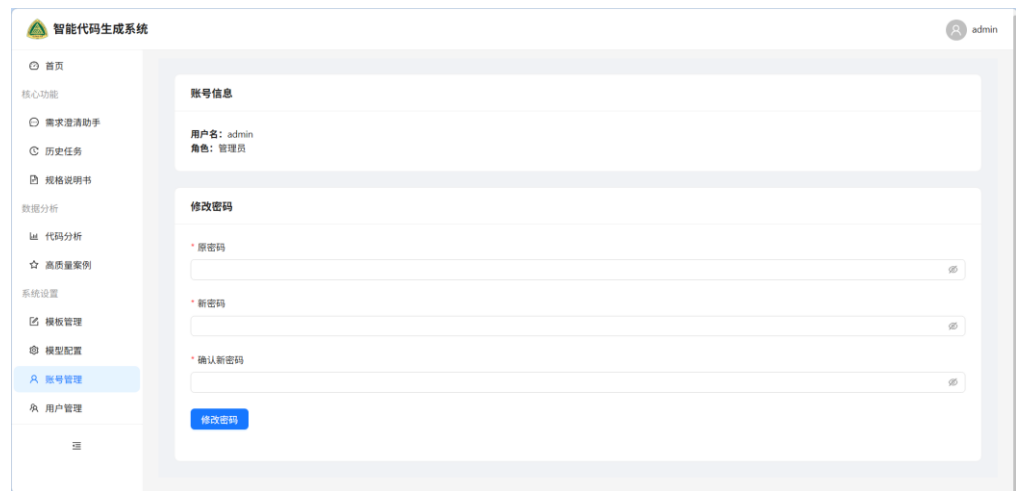


图 5.20 账号管理页面

5.7.8 模型配置与用户管理（管理员专属）

管理员登录后，侧边栏的“系统设置”中会额外显示“模型配置”和“用户管理”菜单。

模型配置：如图 5.21 所示，表格列出所有已配置的模型，包含模型名称、API Base URL、状态（启用/停用）及操作按钮。管理员可通过状态开关快速启用或停用模型，也可点击“编辑”或“删除”按钮进行相应操作。表格上方设有“新增模型”按钮，用于添加新的模型配置。

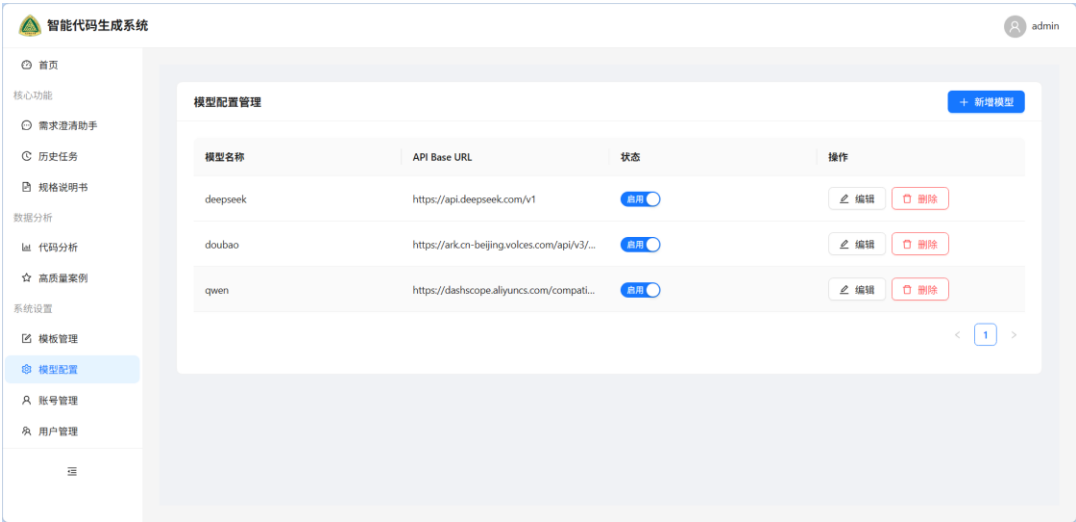


图 5.21 模型配置管理界面

用户管理：如图 5.22 所示，管理员可以查看所有注册用户，表格包含用户名、管理员权限标识、专业权重（百分比）、测评状态。管理员可编辑用户信息（修改用户名或管理员权限），设置用户评分权重（影响该用户在评分环节的权重），重置用户密码，以及删除普通用户。系统会保护最后一个管理员账号不被删除。测评状态反映用户是否已完成所有测评任务（如“已完成”表示已通过测评）。

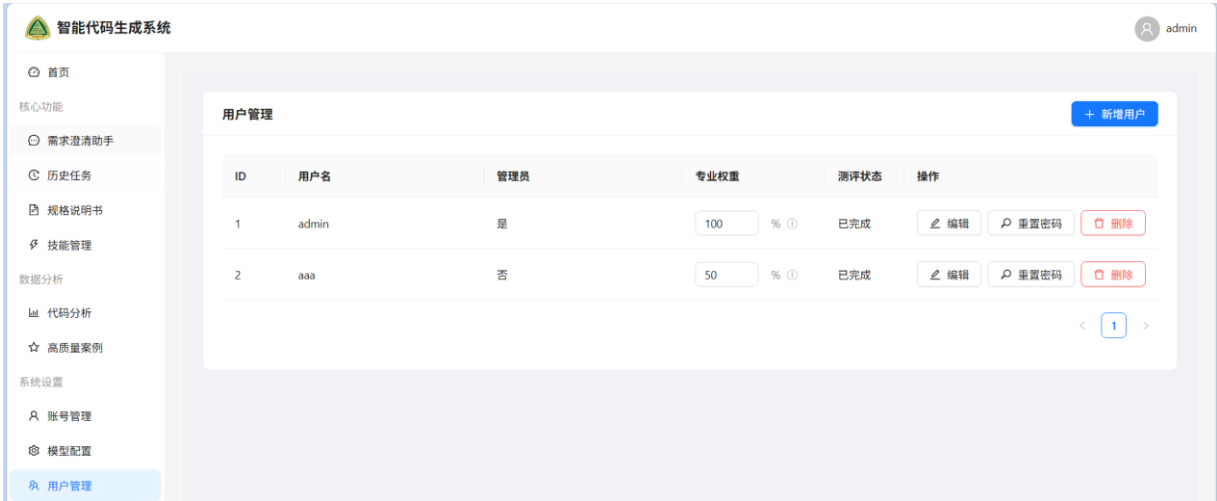


图 5.22 用户管理界面

5.8 结构化需求规约示例

当一轮对话澄清完成时，系统会生成结构化的需求规约。下面是经过完整澄清后得到的一个规约示例，以 JSON 格式存储。如表 5.14 所示。

表 5.14 结构化需求规约示例

字段	内容
original	”写一个函数，对列表进行排序。”
details	{“输入规格”:“整数列表”, “输出规格”:“返回升序排序后的新列表”, “边界条件”:“空列表返回空列表”, “异常处理”:“遇到非整数元素时抛出 TypeError”, “性能约束”:“时间复杂度不超过 O(nlogn)”}

其中 original 字段记录用户最初输入的自然语言需求，details 字段以键值对的形式存储每个澄清维度下用户提供的具体信息。这种结构化的规约既便于后续代码生成模块直接使用，也支持用户回溯和修改。前端在展示规约时，可以将 details 中的内容实时渲染为可读的列表或卡片，增强交互透明性。

5.9 本章小结

本章详细阐述了基于交互式需求澄清的智能代码生成系统的详细设计与实现。首先，介绍了多智能体协作框架的整体结构，明确协调器与五个智能体的交互关系。其次，分别深入分析了四个核心模块的设计原理与实现细节：需求模糊性检测采用“开放式反思+结构化后备”的策略，并设计了优雅的降级机制；澄清问题生成通过独立构造提示词为每个模糊点生成引导性问题；多轮对话管理基于有限状态机实现，创新性地引入了回滚机制保证规约一致性，并通过状态持久化支持会话恢复；代码生成模块支持三种生成模式并内置了安全检测规则。接着，介绍了历史经验学习机制的设计与实现，使系统具备从成功案例中学习的能力。最后，展示了前端核心交互逻辑和用户界面实现，并通过结构化需求规约示例说明了系统的实际运行效果。本章为系统的功能实现提供了完整的技术支撑。

第 6 章 系统测试与评估

6.1 测试需求分析

系统除核心的需求澄清与代码生成外，还包含模型配置管理、澄清模板管理、高质量案例库、规格说明书管理、历史任务追溯、用户管理、代码分析等模块。功能测试需求如表 6.1 所示。

表 6.1 功能需求分析

测试需求	测试需求简介	优先级
用户注册与登录	用户注册、登录、令牌验证、权限区分	1
需求澄清与代码生成	模糊点检测、问题生成、多轮对话、回滚、代码生成	1
自动化测试	可测试性检查、测试代码生成、结果保存与对比	1
技能管理	技能创建、编辑、删除、关联模板、应用技能	2
用户专业测评	测评问卷提交、权重计算、重复测评拦截	2
历史任务追溯	会话列表展示、详情查看、重新评分、删除会话	2
高质量案例库	案例收录、搜索、预览、删除	2
模板管理	自定义澄清模板的增删改查及启停	2
规格说明书管理	自动保存、编辑、删除规格文档	2
模型配置管理	模型增删改查、启用/停用	2
用户管理	用户增删改查、重置密码、权重设置	2
代码分析	评分概览、趋势图、模型对比柱状图、测试通过率对比	3
状态持久化与会话恢复	刷新页面或重新登录后恢复对话状态	2
异常处理	网络错误、API 密钥无效、LLM 调用超时等	2

6.2 实验环境与配置

实验环境及配置如表 6.2 所示。

表 6.2 实验环境及配置

实验环境	实验配置
操作系统	Windows10
CPU	11thGenIntel(R)Core(TM)i7-1165G7@2.80GHz
内存	32GB
编程语言	Python3.10
编译工具	PyCharm2024.2
后端框架	FastAPI+Uvicorn
前端框架	React

6.3 单元测试

单元测试针对每个核心模块的独立功能进行。后端使用 `pytest` 框架，前端使用 `Jest + React Testing Library`。

1) 模糊性检测模块：对 `detect_ambiguity` 函数编写 4 个测试用例（清晰需求、单维度模糊、多维度模糊、空输入）。均在 `pytest` 中通过，如图 6.1 所示。

```
collected 4 items

tests/test_ambiguity_detector.py::test_requirement_with_ambiguity PASSED [ 25%]
tests/test_ambiguity_detector.py::test_single_dimension_ambiguous PASSED [ 50%]
tests/test_ambiguity_detector.py::test_multi_dimension_ambiguous PASSED [ 75%]
tests/test_ambiguity_detector.py::test_empty_input PASSED [100%]

===== 4 passed in 45.03s =====
```

图 6.1 模糊检测模块单元测试

2) 问题生成模块：对 `generate_questions` 函数编写 3 个测试用例（正常模糊点、空列表、维度缺失）。均在 `pytest` 中通过，如图 6.2 所示。

```
collected 3 items

tests/test_question_generator.py::test_generate_questions_normal PASSED [ 33%]
tests/test_question_generator.py::test_generate_questions_empty_list PASSED [ 66%]
tests/test_question_generator.py::test_generate_questions_missing_dimension PASSED [100%]

===== 3 passed in 5.88s =====
```

图 6.2 问题生成模块单元测试

3) 协调器（`Orchestrator`）状态转换：对 `Orchestrator` 类的状态转换编写 5 个测试用例（正常流程-需求清晰、正常流程-需求歧义、正常流程-回答澄清问题、冲突回滚、索引越界）。均在 `pytest` 中通过，如图 6.3 所示。

```
collected 5 items

tests/test_orchestrator.py::test_receive_requirement_clear PASSED [ 20%]
tests/test_orchestrator.py::test_receive_requirement_ambiguous PASSED [ 40%]
tests/test_orchestrator.py::test_receive_answer_normal_flow PASSED [ 60%]
tests/test_orchestrator.py::test_receive_answer_conflict_rollback PASSED [ 80%]
tests/test_orchestrator.py::test_receive_answer_index_out_of_range PASSED [100%]

===== 5 passed in 0.27s =====
```

图 6.3 协调器状态转换单元测试

4) 前端组件测试：

对 CodeGen、History、Cases、ModelConfig 等页面组件使用 Jest + React Testing Library 编写关键交互测试。

测试结果：所有单元测试通过率 100%。

测试分析：单元测试验证了各模块在隔离环境下的正确性，尤其是冲突检测与回滚逻辑在预期条件下均能正确触发，为集成测试奠定了基础。

6.4 集成测试

集成测试重点验证模块间交互、API 接口契约以及前后端数据一致性。

1.核心澄清-生成流程（端到端）

- 1) POST/api/start→创建会话，返回 session_id
- 2) POST/api/detect→发送需求，返回模糊点列表或直接进入澄清
- 3) POST/api/confirm_ambiguities→确认模糊点（用户可增删改），返回第一个澄清问题
- 4) POST/api/answer（循环）→提交答案，直至 ready=True，响应中包含 code 与 explanation
- 5) 验证对话历史正确存储于数据库 SessionRecord 和 SpecDocument 表中。

2.辅助功能模块集成测试

- 1) 模型配置：管理员新增模型后，GET/api/model_configs 立即返回新模型；普通用户请求该 API 只能看到 is_active=True 的模型。
- 2) 模板管理：用户创建模板后，GET/api/templates 仅返回当前用户的模板，不同用户数据隔离。
- 3) 案例库：POST/api/cases/mark 将当前会话代码收录到 HighQualityCase 表；普通用户查看案例列表仅看到自己标记的案例，管理员可见全部。
- 4) 规格文档：代码生成后自动调用 auto_save_spec_document；用户通过 GET/api/spec_documents 获取文档列表，并支持 DELETE 删除。
- 5) 代码分析：GET/api/analysis/overview、/trend、/model_comparison 基于当前用户的 CodeRating 记录返回统计数据，数据隔离正确。

3.回滚与状态持久化

模拟用户回答到第 3 个问题后点击“回滚”，验证 Orchestrator 从 SessionRecord.orchestrator_state 恢复正确状态，且 current_question_index 回退，重新生成代码时不再要求已回答的问题。

4.前后端集成测试

使用 Postman 自动化测试集（共 24 个场景，覆盖所有重要 API），同时配合前端 ReactTestingLibrary 模拟完整用户旅程。所有场景通过，API 平均响应时间约为 3.2 秒（不含 LLM 调用），JSON 数据格式与前端 TypeScript 接口定义一致。

6.5 系统测试

系统测试在完整部署的环境下进行，包含功能测试和性能测试。

6.5.1 功能测试

为全面验证系统功能的正确性，按照模块划分设计详细的测试用例。

1) 用户认证模块

用户认证模块包括注册、登录、修改密码及权限区分。测试用例如表 6.3 所示。

表 6.3 用户认证模块测试用例

用例编号	测试子项	操作步骤	预期结果	实际结果
TC-AUTH-01	用户注册	输入用户名“testuser”，密码“Pass123”，确认密码一致	注册成功，自动登录并跳转首页	符合预期结果
TC-AUTH-02	登录	输入正确用户名密码	登录成功，顶部显示用户名	符合预期结果
TC-AUTH-03	修改密码	原密码正确，新密码“New789”	提示“密码修改成功”，需重新登录	符合预期结果
TC-AUTH-04	权限隔离	普通用户访问 /model-config 路由	前端路由守卫拦截，提示无权限	符合预期结果

2) 需求澄清与代码生成模块

该模块是系统核心，涵盖模糊点识别、多轮对话、回滚、代码生成及保存等子功能。测试用例如表 6.4 所示。

表 6.4 需求澄清与代码生成模块测试用例

用例编号	测试子项	操作步骤	预期结果	实际结果
TC-GEN-01	清晰需求	输入“写一个函数，返回两个整数的和”	不产生澄清问题，直接生成代码	符合预期结果
TC-GEN-02	模糊需求	输入“写一个函数，对列表进行排序”	返回模糊点列表并弹出编辑框	符合预期结果
TC-GEN-03	触发澄清	删除“性能约束”，新增“稳定性要求”	确认后生成的第一个问题基于修改后的列表	符合预期结果
TC-GEN-04	编辑模糊点	问题含“A. 升序 B. 降序”，输入“A”	系统将“A”解析为“升序”存入规约	符合预期结果
TC-GEN-05	字母选项回答	回答完第 2 个问题后点击“回滚”按钮	界面恢复到第 2 个问题的提问状态	符合预期结果
TC-GEN-06	回滚功能	生成代码中包含 # CLARIFIED: 行	前端以黄色背景高亮显示该行	符合预期结果

3) 历史任务追溯模块

历史任务模块支持用户查看已完成会话的详情、对话记录、最终代码以及编辑评分。测试用例如表 6.5 所示。

表 6.5 历史任务追溯模块测试用例

用例编号	测试子项	操作步骤	预期结果	实际结果
TC-HIST-01	查看历史任务列表	完成至少一个会话后进入“历史任务”页面	显示会话记录，包含原始需求、模型、评分	符合预期结果
TC-HIST-02	查看详情	点击某记录的“详情”按钮	弹出模态框，展示完整对话历史、最终代码	符合预期结果
TC-HIST-03	编辑用户评分	点击评分编辑图标，输入 85 分并保存	列表中评分更新，案例库中对应案例同步更新	符合预期结果

4) 高质量案例库模块

案例库模块允许用户手动或自动将高分代码收录，并支持搜索与预览。测试用例如表 6.6 所示。

表 6.6 高质量案例库模块测试用例

用例编号	测试子项	操作步骤	预期结果	实际结果
TC-CASE-01	手动收录案例	代码生成后点击“★手动收录”按钮	提示“已收录”，案例出现在“高质量案例”页面	符合预期结果
TC-CASE-02	自动收录	对生成代码用户评分 ≥85 分	弹窗询问是否收录，确认后自动加入案例库	符合预期结果
TC-CASE-03	搜索与预览	在案例库搜索框输入需求关键词，点击“预览”	列表动态过滤，预览窗口显示完整代码	符合预期结果

5) 模板管理模块

模板管理模块允许用户自定义澄清问题模板，提升模糊点识别的一致性。测试用例如表 6.7 所示。

表 6.7 模板管理模块测试用例

用例编号	测试子项	操作步骤	预期结果	实际结果
TC-TMPL-01	新增模板	填写维度“并发要求”，问题“是否支持多线程？”并保存	模板出现在列表中；下次模糊检测生成该维度问题	符合预期结果
TC-TMPL-02	编辑模板	修改已有模板的默认答案	下次生成澄清问题时默认值生效	符合预期结果
TC-TMPL-03	停用模板	将某个模板的“启用”开关关闭	新的模糊检测不再生成该维度的问题	符合预期结果

6) 模型配置模块（管理员）

模型配置模块仅管理员可见，支持动态增删改查启用的 LLM 模型。测试用例如表 6.8 所示。

表 6.8 模型配置模块测试用例

用例编号	测试子项	操作步骤	预期结果	实际结果
TC-MODEL-01	新增模型	填写名称“gpt-4”、APIKey 等，启用	模型出现在列表中；前端下拉框增加“gpt-4”	符合预期结果
TC-MODEL-02	切换模型生成代码	在下拉框选择“gpt-4”，发送需求并完成澄清	代码由新模型生成，对话历史记录模型名称	符合预期结果
TC-MODEL-03	停用模型	将“gpt-4”停用	前端下拉框不再显示该模型，已有会话不受影响	符合预期结果

7) 用户管理模块（管理员）

用户管理模块支持管理员对普通用户进行创建、编辑、重置密码及删除操作。测试用例如表 6.9 所示。

表 6.9 用户管理模块测试用例

用例编号	测试子项	操作步骤	预期结果	实际结果
TC-USER-01	创建普通用户	输入“alice”/“123456”，非管理员	用户 alice 可正常登录，且无管理员权限	符合预期结果
TC-USER-02	重置密码	为 alice 重置密码为“newpass”	alice 使用新密码登录成功	符合预期结果

TC-USER-03	删除用户	删除非管理员用户	用户被删除，无法登录	符合预期结果
------------	------	----------	------------	--------

8) 规格说明书管理模块

规格说明书模块自动保存每次生成代码时的需求和澄清规约，并支持编辑与删除。测试用例如表 6.10 所示。

表 6.10 规格说明书管理模块测试用例

用例编号	测试子项	操作步骤	预期结果	实际结果
TC-SPEC-01	自动保存	完成代码生成后，进入“规格说明书”页面	自动生成记录，含原始需求、澄清规约 JSON	符合预期结果
TC-SPEC-02	编辑并保存	点击“编辑”，修改标题和澄清规约 JSON	保存后再次查看显示最新内容	符合预期结果
TC-SPEC-03	删除	点击“删除”并确认	记录从列表中消失	符合预期结果

9) 代码分析模块

代码分析模块提供评分概览、趋势图及模型对比柱状图，帮助用户评估代码质量。测试用例如表 6.11 所示。

表 6.11 代码分析模块测试用例

用例编号	测试子项	操作步骤	预期结果	实际结果
TC-ANAL-01	概览数据	完成多个会话并评分后，进入“代码分析”	显示“已评估记录”总数、“整体平均分”	符合预期结果
TC-ANAL-02	趋势图	连续生成多个代码并评分	折线图显示最近 10 次评分走势	符合预期结果
TC-ANAL-03	模型对比柱状图	使用不同模型生成代码并评分	柱状图按模型展示平均分	符合预期结果

10) 异常处理模块

异常处理模块验证系统在异常情况（网络断开、API 密钥错误等）下的鲁棒性。测试用例如表 6.12 所示。

表 6.12 异常处理模块测试用例

用例编号	测试子项	操作步骤	预期结果	实际结果
TC-ERR-01	网络断开	发送需求后断开后端服务	前端提示“处理失败”，界面不崩溃	符合预期结果
TC-ERR-02	APIKey 无效	在模型配置中填入错误 Key，使用该模型生成代码	后端返回 500 错误，前端显示	符合预期结果

11) 会话管理模块

会话管理模块支持用户重置当前对话或删除历史会话记录。测试用例如表 6.13 所示。

表 6.13 会话管理模块测试用例

用例编号	测试子项	操作步骤	预期结果	实际结果
TC-SESSION-01	重置会话	对话过程中点击“新对话”按钮	聊天记录清空, 显示初始欢迎语, 会话 ID 更新	符合预期结果
TC-SESSION-02	删除历史会话	在“历史任务”中点击某会话的“删除”	该会话从列表移除	符合预期结果

12) 自动化测试模块

自动化测试模块支持用户进行一键式轻量测试。测试用例如表 6.14 所示。

表 6.14 自动化测试模块测试用例

用例编号	测试子项	操作步骤	预期结果	实际结果
TC-TEST-01	可测试性检查-正常	代码包含一个纯函数	返回 testable: true	符合预期结果
TC-TEST-02	可测试性检查-含 GUI	代码含 import pygame	返回 testable: false 及原因	符合预期结果
TC-TEST-03	测试代码生成	提供需求“加法函数”及代码	生成的测试代码包含 from solution import * 和 def test_add()	符合预期结果
TC-TEST-04	测试执行通过	对正确的加法函数运行测试	pass_rate=100, experimental_passed=true	符合预期结果
TC-TEST-05	测试执行失败	对缺少输入校验的阶乘函数测试	pass_rate=0, experimental_passed=false	符合预期结果
TC-TEST-06	测试结果保存与展示	完成测试后进入“代码分析”页面	实验组/对照组通过率柱状图显示正确, 历史表格包含记录	符合预期结果

13) 技能管理模块

技能管理模块支持用户进行创建技能库并应用。测试用例如表 6.15 所示。

表 6.15 技能管理模块测试用例

用例编号	测试子项	操作步骤	预期结果	实际结果
TC-SKILL-01	创建技能	填写名称“Web 开发”，从左侧模板列表选择 2 个模板	技能保存成功，关联模板正确	符合预期结果
TC-SKILL-02	编辑技能	修改技能描述，更改关联模板	技能更新，原关联模板被替换	符合预期结果
TC-SKILL-03	应用技能	点击技能“应用”，跳转到代码生成页面，输入需求	系统不再提问技能关联的维度	符合预期结果

14) 用户测评模块

用户测评模块支持用户进行测评并评定专业权重。测试用例如表 6.16 所示。

表 6.16 用户测评模块测试用例

用例编号	测试子项	操作步骤	预期结果	实际结果
TC-ASSESS-01	完成测评	新用户注册后自动跳转测评页，完成所有题目	提交后显示专业权重（如 70%），存入数据库	符合预期结果
TC-ASSESS-02	重复测评拦截	已测评用户再次访问 /assessment	自动重定向到仪表盘	符合预期结果
TC-ASSESS-03	管理员设置权重	管理员进入用户管理，修改某用户的专业权重为 90	用户权重更新，影响后续案例质量分计算	符合预期结果

6.5.2 代码通过率验证（对比实验）

为验证交互式需求澄清机制对代码正确性的提升效果，设计了对比实验。

1) 数据集与任务构造：

实验采用 PURE(Public Requirements)数据集^[61]。该数据集来自于 79 份真实的软件需求规格说明书(SRS)，有 34268 个需求句子，涉及各个领域，被软件需求工程领域的专业人士所广泛使用，是软件需求工程领域公认的高质量基准数据集。pure 需求文本结构完整、语法正确，可用来对需求进行澄清以及产生代码生成任务。从该数据集中选取了 10 个有典型模糊性或者不完整性的需求，构建测试任务集，具体描述如表 6.17 所示。

2) 测试用例设计：

为每个任务独立编写单元测试文件（test_task1.py~test_task10.py）。每个测试文件包含：

正常路径：典型的有效输入及预期输出；

边界条件：端点值、空列表、长度为零等；

异常输入：类型错误、越界值、重复使用等。

判定标准：执行 pytest 后所有断言均通过（returncode==0）视为该任务通过。

3）实验过程：

对照组：将原始需求（task_data.csv 中的 requirement_zh 字段）直接输入 DeepSeek API，参数 temperature=0.2，不进行任何交互。每个任务生成一次代码，保存至 baseline/目录。

实验组：使用本系统，由研究者扮演用户，根据系统提出的澄清问题（如“是否包含端点？”、“默认长度是多少？”）提供标准答案（依据 answers_guide.md），经过多轮问答后生成最终代码，保存至 experimental/目录。

执行测试：运行 run_tests.py 分别对两组代码执行单元测试，统计 Pass@1（首次生成代码的通过比例）。

表 6.17 对照组与实验组代码通过率对比

任务编号	模糊需求描述（关键模糊点）	对照组	实验组
1	比较温度是否超过限值（LO/UO），需明确端点是否视为超过。	通过	通过
2	温度区间与加热/冷却动作的对应关系，缺少 UO 定义及边界处理。	通过	通过
3	随机密码生成，未指定默认长度及是否允许用户指定长度。	失败	通过
4	TAN 一次性密码，未明确过期状态存储与重复使用校验机制。	通过	通过
5	搜索结果排序，未要求稳定排序及相等元素的顺序保持。	通过	通过
6	块移动碰撞检测，未明确“重叠”的数学定义及边接触是否允许。	通过	通过
7	撤销/重做历史管理，未指定历史容量上限及新操作对重做栈的影响。	失败	通过
8	用户查找，未说明匹配方式（大小写敏感/子串/返回单条还是多条）。	失败	通过
9	温度有效性验证，未说明端点是否包含及返回值类型。	通过	通过
10	网站过滤，未明确过滤方向（白名单/黑名单）、空列表行为及域名提取规则。	通过	通过
合计		7/10	10 / 10

	(70.0%)	(100.0%)
--	---------	----------

测试结果与数据分析：

测试结果如图 6.4 所示，对照组 Pass@1=70.0%(7/10)；实验组 Pass@1=100.0%(10/10)，绝对提升 30 个百分点，相对提升 42.9%。

测试分析：由于需求中隐含的约束（如任务 2 的 UO 定义、任务 10 的空列表行为）在 PURE 数据集中已有一定体现，且常见编程惯例可部分弥补，因此对照组也能通过测试。但对于需要精确接口规范（如任务 3 的默认参数）、明确历史行为（如任务 7 的栈清空）或匹配规则细节（如任务 8 的字段命名与模糊匹配）的任务，对照组因缺乏必要澄清而产生偏差，而实验组通过交互式问答准确补全了这些信息，全部成功。

```
PS F:\codegen_experiment> python run_tests.py --group baseline
任务 1: 通过
任务 2: 通过
任务 3: 失败
任务 4: 通过
任务 5: 通过
任务 6: 通过
任务 7: 失败
任务 8: 失败
任务 9: 通过
任务 10: 通过

baseline 组通过率: 70.0% (7/10)
结果已保存到 results\baseline_results.csv
PS F:\codegen_experiment> python run_tests.py --group experimental
任务 1: 通过
任务 2: 通过
任务 3: 通过
任务 4: 通过
任务 5: 通过
任务 6: 通过
任务 7: 通过
任务 8: 通过
任务 9: 通过
任务 10: 通过

experimental 组通过率: 100.0% (10/10)
结果已保存到 results\experimental_results.csv
```

图 6.4 对比实验运行截图

6.5.3 性能测试

用 Python 编写一个多线程脚本，模拟 10 个虚拟用户，对每一个用户从输入需求、回答三个问题、生成代码的过程都进行模拟。经过三次测试（共 30 个会话）之后得到的平均响应时间、CPU 占用率以及内存占用。

测试结果：

- 1) 平均单会话响应时间是 4.1 秒。
- 2) 最大并发时 CPU 占用率不大于 65%，内存占用不大于 488MB。
- 3) 无请求失败或状态错误。

测试分析：系统响应时间低于非功能要求（ ≤ 50 秒），大模型单次调用通常为 20 - 40 秒，本系统通过多轮交互未明显增加延迟。资源占用低，可稳定支持小批量（10 人级）同时在线。

6.6 测试结果与结论

测试结论：

- 1) 功能完整性：全部核心模块（模糊检测、多轮对话、代码生成、历史追溯、案例库、模板管理、规格文档、模型配置、用户管理等）运行正常。
- 2) 可靠性与体验：回滚机制稳定，提升容错能力。
- 3) 对比实验：交互式需求澄清使 Pass@1 从 70.0%提升至 100.0%，绝对提升 30 个百分点，相对提升 42.9%。
- 4) 非功能指标：响应时间、并发、异常处理均符合要求。

6.7 本章小结

本章对基于交互式需求澄清的智能代码生成系统进行了全面的测试与评估。通过单元测试、集成测试和系统测试（含功能测试、性能测试），验证了系统的正确性、稳定性和可用性，对比实验结果表明，本系统提升了模糊需求场景下的代码生成质量，验证了交互式需求澄清机制和“人在回路”设计理念的有效性。

第 7 章 总结与展望

7.1 主要工作

本文针对目前智能代码生成技术处理模糊需求时存在的不足，设计并实现了一个交互式需求澄清的智能代码生成系统。完成的主要工作及存在的不足如下：

第一，关键技术研究与借鉴。针对目前的 `chatcoder`、`clarifygpt` 等最新的交互式需求澄清框架的研究结果进行整理，并吸取其中所包含的一些结构化的描述方式以及代码一致性等方法，进而完成系统的规划工作。

第二，系统核心模块的设计和实现。设计并实现需求模糊性检测、澄清问题生成、多轮对话管理、代码生成与解释、自动化测试对比、技能管理、用户专业测评等主要模块。模糊检测模块采用开放式反思、结构化核对两种提示方式，对话管理模块用有限状态机来达成回滚和状态持久化的目的，从而保障整个交互过程是稳定的、有条理的。

第三，原型系统的开发与集成。使用前后端分离的架构来开发出一个带有网页对话界面的原型系统。前端给用户提供直观交互，后端将大模型 API 以及测试执行器融合起来，从而完成需求输入、交互确认、代码生成、自动化测试比较整个自动化流程。

第四，实验验证与效果评估。创建出一个有 10 个典型模糊场景的数据集，然后对它做了严格的对比实验。从交互式需求澄清之后得到的结果来看，系统代码通过率（Pass@1）由原来的直接生成的 70.0% 提高到了现在的 100.0%，说明系统的设计是有效的。

与此同时，本系统仍存在以下两方面的主要不足：

不足一：历史经验学习机制比较初级。目前的历史经验只能依靠关键词重叠来达到相似检索的目的，以文本的形式作为参考提示注入上下文，不能达到语义级别的相似匹配，也不能将以前建立好的澄清规约自动生成的方式应用到当前的对话当中，造成经验复用的智能化程度较低。

不足二，交互策略没有个性化的、自适应的。系统澄清提问的流程是一成不变的，对于所有的用户的提问顺序和问题表述方式都按照相同的规则来设置，没有考虑到用户的前言回答习惯和专业程度等因素，从而造成交互的低效。。

7.2 工作展望

未来可以从两个方面进行改进和提高：

第一，增强历史经验学习的深度与智能化，实现语义检索与自动预填充。

利用向量数据库（Chroma、Milvus）对历史成功的案例中所包含的澄清规约以及代码进行语义嵌入，从而完成语义相似度的精确检索，以此来突破当前仅依靠关键词进行匹配的方式。因此，把高相似的案例完全地填充到当前会话里，用户只要对重复的部分进行确认或者微调就可以大幅度减少重复输入，从而提高经验复用的智能化程度。还可以设计消融实验，定量地测量高层次大模型（GPT-4）作为历史经验生成器的独立收益。

第二，实现个性化与自适应的交互策略，提升澄清效率与用户体验。

根据用户以前给出的提示速度、答案的好坏、修改次数等行为特点变化情况来调整澄清问题的难易程度、选项颗粒度、提问次序等，从而克服目前固定流程中存在不足的问题。使用用户满意度作为奖励信号，在用户满意度的基础上使用强化学习来使系统自动地学习出最佳提问时间、提问方式，从而给各个专业水平的用户提供个性化的澄清渠道，提高用户的交互体验。

参考文献

- [1] SUN J, LIAO Q V, MULLER M, et al. Investigating explainability of generative AI for code through scenario-based design [C]//Proceedings of the 27th International Conference on Intelligent User Interfaces. New York: ACM, 2022: 212-224.
- [2] MEINCKE L, MOLLICK E, MOLLICK L, et al. Prompting science report 1: prompt engineering is complicated and contingent [OL]. (2025-03-07) [2026-04-18]. <https://arxiv.org/abs/2503.04818>.
- [3] VAITHILINGAM P, ZHANG T, GLASSMAN E L. Expectation vs. experience: evaluating the usability of code generation tools powered by large language models [C]//Extended Abstracts of the 2022 CHI Conference on Human Factors in Computing Systems. New York: ACM, 2022: 1-7.
- [4] WEIDMANN N, YIGITBAS E, ANJORIN A, et al. Human-in-the-loop large-scale model transformations with the VICToRy debugger [J]. Journal of Object Technology, 2022, 21(2): 3: 1-15.
- [5] 张缨斌,吴若乔,何雨轩,等.感知情境与人在回路的智能教育——《人工智能与教育的未来: 见解与提议》要点与反思[J].开放教育研究,2023,29(4):11-20.
- [6] CLEMMENSEN T, MOGHADDAM M T, NØRBJERG J. Cyber-physical systems with human-in-the-loop: a systematic review of socio-technical perspectives [J]. Journal of Systems and Software, 2025, 226: 112348.
- [7] 孙佺,李国良.人在回路的数据融合系统[J].计算机学报,2022,45(03):654-668.
- [8] WU T, TERRY M, CAI C J. AI chains: transparent and controllable human-AI interaction by chaining large language model prompts [C]//Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems. New Orleans: ACM, 2022: 1-22.
- [9] AMNA A R, WAUTELER Y, POELMANS S, et al. The AmbiTRUS framework for identifying potential ambiguity in user stories [J]. Journal of Systems and Software, 2025, 223: 112357.
- [10] SAN MARTIN A, KILDAL J, LAZKANO E. An analysis of the role of different levels of exchange of explicit information in human-robot cooperation [J]. Frontiers in Robotics and AI, 2025, 12: 1511619.

- [11] SHAKERI HOSSEIN ABAD Z, MOAZZAM S, LO C, et al. Loud and interactive paper prototyping in requirements elicitation: what is it good for? [C]//Proceedings of the 2018 IEEE 7th International Workshop on Empirical Requirements Engineering. Banff: IEEE, 2018: 16-23.
- [12] CLARK H H, SCHAEFER E F. Contributing to Discourse[J]. Cognitive Science, 1989, 13(2): 259-294. DOI: 10.1207/s15516709cog1302_7. [2026-04-18]. https://doi.org/10.1207/s15516709cog1302_7.
- [13] GRICE H P. Logic and conversation[M]//COLE P, MORGAN J L. Syntax and Semantics, Vol. 3: Speech Acts. New York: Academic Press, 1975: 41-58.
- [14] SPERBER D, WILSON D. Relevance: Communication and Cognition[M]. 2nd ed. Oxford: Blackwell, 1995.
- [15] WANG J, QIANG W, SONG Z, et al. Learning to think: information-theoretic reinforcement fine-tuning for LLMs [OL]. (2025-05-18) [2026-04-18]. <https://arxiv.org/abs/2505.10425>.
- [16] PAN B, ZHAO L. Can past experience accelerate LLM reasoning? [OL]. (2025-05-30) [2026-04-18]. <https://arxiv.org/abs/2505.20643>.
- [17] VASWANI A, SHAZEER N, PARMAR N, et al. Attention is all you need [C]//Proceedings of the 31st International Conference on Neural Information Processing Systems. Long Beach: Curran Associates Inc., 2017: 5998-6008.
- [18] RADFORD A, NARASIMHAN K, SALIMANS T, et al. Improving language understanding by generative pre-training [OL]. (2018-06-11) [2026-04-18]. <https://openai.com/research/language-unsupervised>.
- [19] RADFORD A, WU J, CHILD R, et al. Language models are unsupervised multitask learners [OL]. (2019-02-14) [2026-04-18]. <https://openai.com/research/better-language-models>.
- [20] BROWN T, MANN B, RYDER N, et al. Language models are few-shot learners [C]//Proceedings of the 34th International Conference on Neural Information Processing Systems. Vancouver: Curran Associates Inc., 2020: 1877-1901.
- [21] CHEN M, TWOREK J, JUN H, et al. Evaluating large language models trained on code [OL]. (2021-07-07) [2026-04-18]. <https://arxiv.org/abs/2107.03374>.
- [22] OPENAI. GPT-4 technical report [OL]. (2023-03-15) [2026-04-18]. <https://arxiv.org/abs/2303.08774>.

-
- [23] POESIA G, POLOZOV O, LE V, et al. Synchromesh: reliable code generation from pre-trained language models [C]//Proceedings of the 10th International Conference on Learning Representations. Online: ICLR, 2022.
- [24] MEINCKE L, PROKSCH S, NADI S. On the impact of natural language description ambiguity on code generation [C]//Proceedings of the 31st IEEE/ACM International Conference on Program Comprehension. Melbourne: IEEE, 2023: 132-143.
- [25] LIU J, XIA C S, WANG Y, et al. Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation [C]//Proceedings of the 37th International Conference on Neural Information Processing Systems. New Orleans: NeurIPS, 2023: 49105-49125.
- [26] PEARCE H, AHMAD B, TAN B, et al. Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions [C]//Proceedings of the 2022 IEEE Symposium on Security and Privacy. San Francisco: IEEE, 2022: 754-768.
- [27] KAMSTIES E, BERRY D M, PAECH B. Detecting ambiguities in requirements documents using inspections [C]//Proceedings of the 4th IEEE International Symposium on Requirements Engineering. Limerick: IEEE, 1999: 148-157.
- [28] FEMMER H, FERNÁNDEZ D M, WAGNER S, et al. Rapid requirements checks with requirements smells: two case studies [C]//Proceedings of the 1st International Workshop on Rapid Continuous Software Engineering. Hyderabad: ACM, 2014: 10-19.
- [29] AMNA A R, POELS G. AmbiTRUS: a framework for ambiguity detection and resolution in user stories [J]. Information and Software Technology, 2024, 167: 107362.
- [30] DOBSON G, SAWYER P. Revisiting the ontology of requirements [C]//Proceedings of the 17th International Working Conference on Requirements Engineering: Foundation for Software Quality. Essen: Springer, 2011: 136-151.
- [31] ARANDA J, ERNST N, EASTERBROOK S. Framework for understanding patterns of socio-technical incongruence in requirements engineering [C]//Proceedings of the 15th IEEE International Requirements Engineering Conference. Delhi: IEEE, 2007: 243-252.

- [32] CARPINETO C, ROMANO G. A survey of automatic query expansion in information retrieval [J]. ACM Computing Surveys, 2012, 44(1): 1-50.
- [33] PATIL S G, BANSAL M, KULKARNI N. Specify by example: using concrete examples to disambiguate program specifications [C]//Proceedings of the 2021 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software. Chicago: ACM, 2021: 15-30.
- [34] CHRISTAKOPOULOU K, RADLINSKI F, HOFMANN K. Towards conversational recommender systems [C]//Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. San Francisco: ACM, 2016: 815-824.
- [35] ROSS S, BAGNELL J A. Efficient reductions for imitation learning [C]//Proceedings of the 13th International Conference on Artificial Intelligence and Statistics. Sardinia: JMLR, 2010: 661-668.
- [36] HOULSBY N, HUSZÁR F, GHAHRAMANI Z, et al. Bayesian active learning for classification and preference learning [OL]. (2011-12-22) [2026-04-18]. <https://arxiv.org/abs/1112.5745>.
- [37] HENDERSON M, THOMSON B, WILLIAMS J D. The second dialog state tracking challenge [C]//Proceedings of the 15th Annual Meeting of the Special Interest Group on Discourse and Dialogue. Philadelphia: ACL, 2014: 263-272.
- [38] RASTOGI A, ZANG X, SUNKARA S, et al. Towards scalable multi-domain conversational agents: the schema-guided dialogue dataset [C]//Proceedings of the 34th AAAI Conference on Artificial Intelligence. New York: AAAI, 2020: 8689-8696.
- [39] WANG Z, LI J, LI G, et al. ChatCoder: chat-based refine requirement improves LLMs' code generation [OL]. (2023-11-01) [2026-05-18]. <https://arxiv.org/abs/2311.00272>.
- [40] MU F, SHI L, WANG S, et al. ClarifyGPT: empowering LLM-based code generation with intention clarification [OL]. (2023-10-17) [2026-04-18]. <https://arxiv.org/abs/2310.10996>.
- [41] AAMODT A, PLAZA E. Case-based reasoning: foundational issues, methodological variations, and system approaches [J]. AI Communications, 1994, 7(1): 39-59.

-
- [42] LI J, ZHAO Y F, LI Y M, LI G, JIN Z. AceCoder: An Effective Prompting Technique Specialized in Code Generation [J/OL]. *ACM Transactions on Software Engineering and Methodology*, 2024, 33(8): 1-26. [2026-04-18]. <https://doi.org/10.1145/3675395>.
- [43] JIANG X, DONG Y H, TAO Y D, LIU H Y, JIN Z, JIAO W P, LI G. ROCODE: Integrating Backtracking Mechanism and Program Analysis in Large Language Models for Code Generation [C] // *Proceedings of the 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. Ottawa, ON, Canada: IEEE, 2025: 334-346. DOI: 10.1109/ICSE55302.2025.00098. [2026-04-18]. <https://doi.org/10.1109/ICSE55302.2025.00098>.
- [44] SCHÄFER M, NADI S, EGHBALI A, et al. TestPilot: an LLM-based approach to generate unit tests for Java [OL]. (2023-10-12) [2026-04-20]. <https://arxiv.org/abs/2310.07800>.
- [45] LEMIEUX C, INALA J P, LAHIRI S K, et al. CodaMosa: escaping coverage plateaus in test generation with pre-trained large language models [C]//*Proceedings of the 45th International Conference on Software Engineering*. Melbourne: IEEE, 2023: 919-931.
- [46] LI J, WANG S, DANG Y, et al. TitanFuzz: fuzzing deep learning compilers with large language models [C]//*Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*. Seattle: ACM, 2023: 202-214.
- [47] RAFAILOV R, SHARMA A, MITCHELL E, et al. Direct preference optimization: your language model is secretly a reward model [C]//*Proceedings of the 37th International Conference on Neural Information Processing Systems*. New Orleans: NeurIPS, 2023: 53728-53741.
- [48] QIAN C, LIU W, LIU H, et al. ChatDev: communicative agents for software development [OL]. (2023-07-16) [2026-04-20]. <https://arxiv.org/abs/2307.07924>.
- [49] HONG S, ZHENG X, CHEN J, et al. MetaGPT: meta programming for multi-agent collaborative framework [OL]. (2023-08-01) [2026-04-20]. <https://arxiv.org/abs/2308.00352>.
- [50] DONG Y, JIANG X, JIN Z, et al. Self-collaboration code generation via ChatGPT [OL]. (2023-04-16) [2026-04-20]. <https://arxiv.org/abs/2304.07590>.

- [51] AMERSHI S, WELD D, VORVOREANU M, et al. Guidelines for human-AI interaction [C]//Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems. Glasgow: ACM, 2019: 1-13.
- [52] CLEMMENSEN T, HERTZUM M, HORNBÆK K, et al. Human-in-the-loop design and evaluation of interactive machine learning [J]. Behaviour & Information Technology, 2020, 39(5): 543-560.
- [53] MOZANNAR H, LANG H, WEI D, et al. When to ask for help: proactive interventions in autonomous agents [OL]. (2022-12-05) [2026-04-20]. <https://arxiv.org/abs/2212.02443>.
- [54] YOUNG S, GAŠIĆ M, THOMSON B, et al. POMDP-based statistical spoken dialog systems: a review [J]. Proceedings of the IEEE, 2013, 101(5): 1160-1179.
- [55] FIELDING R T. Architectural styles and the design of network-based software architectures [D]. Irvine: University of California, Irvine, 2000.
- [56] SCHUBERT L, WEIBERT A, WULF V. How novices and experts interact with conversational AI [C]//Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems. Glasgow: ACM, 2019: 1-15.
- [57] LUBANOVIC B. FastAPI: modern Python web development [M]. Sebastopol: O'Reilly Media, 2022.
- [58] KREBS R, MOMM C, KOUNEV S. Architectural concerns in multi-tenant SaaS applications [C]//Proceedings of the 2nd International Conference on Cloud Computing and Services Science. Porto: SciTePress, 2012: 426-431.
- [59] JONES M, BRADLEY J, SAKIMURA N. JSON Web Token (JWT) [S]. RFC 7519, IETF, 2015.
- [60] KALISKI B. PKCS#5: Password-Based Cryptography Specification Version 2.0 [S]. RFC 2898, IETF, 2000.
- [61] FERRARI A, SPAGNOLO G O, GNESI S. PURE: A Dataset of Public Requirements Documents [C] // 25th IEEE International Requirements Engineering Conference (RE 2017). Lisbon, Portugal: IEEE, 2017: 502-505. DOI: 10.1109/RE.2017.29. [2026-05-18]. <https://doi.org/10.1109/RE.2017.29>.

致谢

本论文的顺利完成，离不开导师的悉心指导和各位同学、朋友的无私帮助。在此，谨向所有给予我支持的人致以最诚挚的谢意。

首先，衷心感谢我的指导老师黄江平老师。从课题选题、系统方案设计到论文撰写，老师始终以严谨的学术态度和耐心的指导帮助我克服重重困难。老师对交互式需求澄清和代码生成领域的深刻理解，为我指明了研究方向；在系统核心模块（如回滚机制、状态机设计、多智能体协作）的实现过程中，老师提出了许多关键性的修改建议，使我能够突破技术瓶颈，最终完成一个可稳定运行的原型系统。老师不仅在学术上言传身教，更在治学精神和做事态度上给我树立了榜样，使我受益良多。

感谢在论文调研阶段给予我启发的各位学者，本人所引用的 ChatCoder、ClarifyGPT、TraceCoder 等前沿研究成果，为本系统的设计与实现提供了重要的理论基础和技术借鉴，在此向这些研究团队表示敬意。

感谢同窗好友，在单元测试、集成测试和测试数据集的构建中提供了大量帮助，并对部分代码逻辑提出了优化建议，使系统更加健壮。感谢室友在论文修改期间的鼓励与陪伴，让我始终保持积极的心态。感谢参与用户访谈和系统试用体验的几位开发者朋友，你们的反馈帮助我进一步完善了系统的交互设计和功能细节。

最后，感谢学校及学院提供的良好学习环境和资源支持，使我能够顺利完成本课题的研究与论文撰写。

再次向所有关心、帮助和支持我的人表示最诚挚的感谢！